

Business Case Delhivery

August 3, 2025

0.0.1 About Delhivery:

Delhivery is the largest and fastest-growing fully integrated player in India by revenue in Fiscal 2021. They aim to build the operating system for commerce, through a combination of world-class infrastructure, logistics operations of the highest quality, and cutting-edge engineering and technology capabilities. The Data team builds intelligence and capabilities using this data that helps them to widen the gap between the quality, efficiency, and profitability of their business versus their competitors

0.0.2 From Delhivery's Perspective:

- Delhivery aims to establish itself as the premier player in the logistics industry. This case study is of paramount importance as it aligns with the company's core objectives and operational excellence.
- It provides a practical framework for understanding and processing data, which is integral to their operations. By leveraging data engineering pipelines and data analysis techniques, Delhivery can achieve several critical goals.
- First, it allows them to ensure data integrity and quality by addressing missing values and structuring the dataset appropriately.
- Second, it enables the extraction of valuable features from raw data, which can be utilized for building accurate forecasting models.
- Moreover, it facilitates the identification of patterns, insights, and actionable recommendations crucial for optimizing their logistics operations.
- By conducting hypothesis testing and outlier detection, Delhivery can refine their processes and further enhance the quality of service they provide.

0.0.3 Dataset

0.0.4 Column Profiling:

- 1.data - tells whether the data is testing or training data
- 2.trip_creation_time – Timestamp of trip creation
- 3.route_schedule_uuid – Unique ID for a particular route schedule
- 4.route_type – Transportation type

- a. FTL – Full Truck Load: FTL shipments get to the destination sooner, as the truck is making no other pickups or drop-offs along the way
- b. Carting: Handling system consisting of small vehicles (carts)

5.trip_uid - Unique ID given to a particular trip (A trip may include different source and destination centers)

6.source_center - Source ID of trip origin

7.source_name - Source Name of trip origin

8.destination_center - Destination ID

9.destination_name - Destination Name

10.od_start_time - Trip start time

11.od_end_time - Trip end time

12.start_scan_to_end_scan - Time taken to deliver from source to destination

13.is_cutoff - Unknown field

14.cutoff_factor - Unknown field

15.cutoff_timestamp - Unknown field

16.actual_distance_to_destination - Distance in kms between source and destination warehouse

17.actual_time - Actual time taken to complete the delivery (Cumulative)

18.osrm_time - An open-source routing engine time calculator which computes the shortest path between points in a given map (Includes usual traffic, distance through major and minor roads) and gives the time (Cumulative)

19.osrm_distance - An open-source routing engine which computes the shortest path between points in a given map (Includes usual traffic, distance through major and minor roads) (Cumulative)

20.factor - Unknown field

21.segment_actual_time - This is a segment time. Time taken by the subset of the package delivery

22.segment_osrm_time - This is the OSRM segment time. Time taken by the subset of the package delivery

23.segment_osrm_distance - This is the OSRM distance. Distance covered by subset of the package delivery

24.segment_factor - Unknown field

0.0.5 Importing Required Libraries:

```
[1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib as mpl
```

```
import matplotlib.pyplot as plt
import scipy.stats as spy
```

```
[2]: import warnings
warnings.simplefilter('ignore')
```

0.0.6 Loading the Dataset:

```
[6]: df = pd.read_csv("delhivery_data.csv")
df.head()
```

```
[6]:      data      trip_creation_time \
0  training  2018-09-20 02:35:36.476840
1  training  2018-09-20 02:35:36.476840
2  training  2018-09-20 02:35:36.476840
3  training  2018-09-20 02:35:36.476840
4  training  2018-09-20 02:35:36.476840

      route_schedule_uuid route_type \
0  thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...  Carting
1  thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...  Carting
2  thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...  Carting
3  thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...  Carting
4  thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...  Carting

      trip_uuid source_center      source_name \
0  trip-153741093647649320  IND388121AAA  Anand_VUNagar_DC (Gujarat)
1  trip-153741093647649320  IND388121AAA  Anand_VUNagar_DC (Gujarat)
2  trip-153741093647649320  IND388121AAA  Anand_VUNagar_DC (Gujarat)
3  trip-153741093647649320  IND388121AAA  Anand_VUNagar_DC (Gujarat)
4  trip-153741093647649320  IND388121AAA  Anand_VUNagar_DC (Gujarat)

      destination_center      destination_name \
0      IND388620AAB  Khambhat_MotvdDPP_D (Gujarat)
1      IND388620AAB  Khambhat_MotvdDPP_D (Gujarat)
2      IND388620AAB  Khambhat_MotvdDPP_D (Gujarat)
3      IND388620AAB  Khambhat_MotvdDPP_D (Gujarat)
4      IND388620AAB  Khambhat_MotvdDPP_D (Gujarat)

      od_start_time ...      cutoff_timestamp \
0  2018-09-20 03:21:32.418600 ...      2018-09-20 04:27:55
1  2018-09-20 03:21:32.418600 ...      2018-09-20 04:17:55
2  2018-09-20 03:21:32.418600 ...  2018-09-20 04:01:19.505586
3  2018-09-20 03:21:32.418600 ...      2018-09-20 03:39:57
4  2018-09-20 03:21:32.418600 ...      2018-09-20 03:33:55

      actual_distance_to_destination  actual_time  osrm_time  osrm_distance \
```

0		10.435660	14.0	11.0	11.9653
1		18.936842	24.0	20.0	21.7243
2		27.637279	40.0	28.0	32.5395
3		36.118028	62.0	40.0	45.5620
4		39.386040	68.0	44.0	54.2181

	factor	segment_actual_time	segment_osrm_time	segment_osrm_distance	\
0	1.272727	14.0	11.0	11.9653	
1	1.200000	10.0	9.0	9.7590	
2	1.428571	16.0	7.0	10.8152	
3	1.550000	21.0	12.0	13.0224	
4	1.545455	6.0	5.0	3.9153	

	segment_factor
0	1.272727
1	1.111111
2	2.285714
3	1.750000
4	1.200000

[5 rows x 24 columns]

0.0.7 What is the shape of the loaded dataset ?

```
[7]: df.shape
```

```
[7]: (144867, 24)
```

0.0.8 What are the columns present in the dataset?

```
[8]: df.columns
```

```
[8]: Index(['data', 'trip_creation_time', 'route_schedule_uuid', 'route_type',
        'trip_uuid', 'source_center', 'source_name', 'destination_center',
        'destination_name', 'od_start_time', 'od_end_time',
        'start_scan_to_end_scan', 'is_cutoff', 'cutoff_factor',
        'cutoff_timestamp', 'actual_distance_to_destination', 'actual_time',
        'osrm_time', 'osrm_distance', 'factor', 'segment_actual_time',
        'segment_osrm_time', 'segment_osrm_distance', 'segment_factor'],
        dtype='object')
```

0.0.9 What is the datatype of the columns ?

```
[9]: df.dtypes
```

```
[9]: data                                object
    trip_creation_time                    object
```

```

route_schedule_uuid      object
route_type               object
trip_uuid               object
source_center            object
source_name              object
destination_center       object
destination_name         object
od_start_time            object
od_end_time              object
start_scan_to_end_scan   float64
is_cutoff                bool
cutoff_factor            int64
cutoff_timestamp         object
actual_distance_to_destination float64
actual_time              float64
osrm_time                float64
osrm_distance            float64
factor                   float64
segment_actual_time      float64
segment_osrm_time        float64
segment_osrm_distance    float64
segment_factor           float64
dtype: object

```

0.0.10 Basic Information about the Dataset

```
[10]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144867 entries, 0 to 144866
Data columns (total 24 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   data                                  144867 non-null object
 1   trip_creation_time                    144867 non-null object
 2   route_schedule_uuid                  144867 non-null object
 3   route_type                           144867 non-null object
 4   trip_uuid                            144867 non-null object
 5   source_center                        144867 non-null object
 6   source_name                          144574 non-null object
 7   destination_center                   144867 non-null object
 8   destination_name                     144606 non-null object
 9   od_start_time                        144867 non-null object
10   od_end_time                          144867 non-null object
11   start_scan_to_end_scan                144867 non-null float64
12   is_cutoff                            144867 non-null bool
13   cutoff_factor                        144867 non-null int64
14   cutoff_timestamp                     144867 non-null object

```

```

15  actual_distance_to_destination  144867 non-null  float64
16  actual_time                    144867 non-null  float64
17  osrm_time                      144867 non-null  float64
18  osrm_distance                  144867 non-null  float64
19  factor                        144867 non-null  float64
20  segment_actual_time            144867 non-null  float64
21  segment_osrm_time              144867 non-null  float64
22  segment_osrm_distance          144867 non-null  float64
23  segment_factor                 144867 non-null  float64
dtypes: bool(1), float64(10), int64(1), object(12)
memory usage: 25.6+ MB

```

0.0.11 Dropping unknown fields

```

[11]: unknown_fields = ['is_cutoff', 'cutoff_factor', 'cutoff_timestamp', 'factor',
    ↪ 'segment_factor']
df = df.drop(columns = unknown_fields)

```

0.0.12 How many unique entries present in each column ?

```

[12]: for i in df.columns:
    print(f"Unique entries for column {i:<30} = {df[i].nunique()}")

```

```

Unique entries for column data = 2
Unique entries for column trip_creation_time = 14817
Unique entries for column route_schedule_uuid = 1504
Unique entries for column route_type = 2
Unique entries for column trip_uuid = 14817
Unique entries for column source_center = 1508
Unique entries for column source_name = 1498
Unique entries for column destination_center = 1481
Unique entries for column destination_name = 1468
Unique entries for column od_start_time = 26369
Unique entries for column od_end_time = 26369
Unique entries for column start_scan_to_end_scan = 1915
Unique entries for column actual_distance_to_destination = 144515
Unique entries for column actual_time = 3182
Unique entries for column osrm_time = 1531
Unique entries for column osrm_distance = 138046
Unique entries for column segment_actual_time = 747
Unique entries for column segment_osrm_time = 214
Unique entries for column segment_osrm_distance = 113799

```

0.0.13 For all those columns where number of unique entries is 2, converting the datatype of columns to category

```
[13]: df['data'] = df['data'].astype('category')
      df['route_type'] = df['route_type'].astype('category')
```

```
[14]: floating_columns = ['actual_distance_to_destination', 'actual_time',
      ↪ 'osrm_time', 'osrm_distance',
      ↪ 'segment_actual_time', 'segment_osrm_time',
      ↪ 'segment_osrm_distance']
      for i in floating_columns:
          print(df[i].max())
```

```
1927.4477046975032
4532.0
1686.0
2326.1991000000003
3051.0
1611.0
2191.4037000000003
```

0.0.14 We can update the datatype to float32 since the maximum value entry is small

```
[16]: for i in floating_columns:
      df[i] = df[i].astype('float32')
```

0.0.15 Updating the datatype of the datetime columns

```
[18]: datetime_columns = ['trip_creation_time', 'od_start_time', 'od_end_time']
      for i in datetime_columns:
          df[i] = pd.to_datetime(df[i])
```

```
[19]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144867 entries, 0 to 144866
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   data                                  144867 non-null  category
1   trip_creation_time                    144867 non-null  datetime64[ns]
2   route_schedule_uuid                  144867 non-null  object
3   route_type                           144867 non-null  category
4   trip_uuid                            144867 non-null  object
5   source_center                        144867 non-null  object
6   source_name                          144574 non-null  object
7   destination_center                   144867 non-null  object
8   destination_name                     144606 non-null  object
```

```

9    od_start_time          144867 non-null  datetime64[ns]
10   od_end_time            144867 non-null  datetime64[ns]
11   start_scan_to_end_scan  144867 non-null  float64
12   actual_distance_to_destination 144867 non-null  float32
13   actual_time            144867 non-null  float32
14   osrm_time              144867 non-null  float32
15   osrm_distance          144867 non-null  float32
16   segment_actual_time    144867 non-null  float32
17   segment_osrm_time      144867 non-null  float32
18   segment_osrm_distance  144867 non-null  float32
dtypes: category(2), datetime64[ns](3), float32(7), float64(1), object(6)
memory usage: 15.2+ MB

```

Earlier the dataset was using 25.6+ MB of memory but now it has been reduced to 15.2 + MB. Around 40.63 % reduction in the memory usage.

0.0.16 What is the time period for which the data is given ?

```
[20]: df['trip_creation_time'].min(), df['od_end_time'].max()
```

```
[20]: (Timestamp('2018-09-12 00:00:16.535741'),
      Timestamp('2018-10-08 03:00:24.353479'))
```

0.1 1. Basic data cleaning and exploration:

0.1.1 Handling missing values in the data

Is there any null values present in the dataset ?

```
[21]: np.any(df.isnull())
```

```
[21]: np.True_
```

0.1.2 What is the number of null values present in each column ?

```
[22]: df.isnull().sum()
```

```
[22]: data
trip_creation_time      0
route_schedule_uuid     0
route_type              0
trip_uuid              0
source_center           0
source_name            293
destination_center      0
destination_name        261
od_start_time           0
od_end_time             0
start_scan_to_end_scan  0
```

```

actual_distance_to_destination    0
actual_time                      0
osrm_time                        0
osrm_distance                    0
segment_actual_time              0
segment_osrm_time                0
segment_osrm_distance            0
dtype: int64

```

```

[23]: missing_source_name = df.loc[df['source_name'].isnull(), 'source_center'].
      ↪unique()
      missing_source_name

```

```

[23]: array(['IND342902A1B', 'IND577116AAA', 'IND282002AAD', 'IND465333A1B',
            'IND841301AAC', 'IND509103AAC', 'IND126116AAA', 'IND331022A1B',
            'IND505326AAB', 'IND852118A1B'], dtype=object)

```

```

[24]: for i in missing_source_name:
      unique_source_name = df.loc[df['source_center'] == i, 'source_name'].
      ↪unique()
      if pd.isna(unique_source_name):
          print("Source Center :", i, "-" * 10, "Source Name :", 'Not Found')
      else:
          print("Source Center :", i, "-" * 10, "Source Name :",
      ↪unique_source_name)

```

```

Source Center : IND342902A1B ----- Source Name : Not Found
Source Center : IND577116AAA ----- Source Name : Not Found
Source Center : IND282002AAD ----- Source Name : Not Found
Source Center : IND465333A1B ----- Source Name : Not Found
Source Center : IND841301AAC ----- Source Name : Not Found
Source Center : IND509103AAC ----- Source Name : Not Found
Source Center : IND126116AAA ----- Source Name : Not Found
Source Center : IND331022A1B ----- Source Name : Not Found
Source Center : IND505326AAB ----- Source Name : Not Found
Source Center : IND852118A1B ----- Source Name : Not Found

```

```

[26]: for i in missing_source_name:
      unique_destination_name = df.loc[df['destination_center'] == i,
      ↪'destination_name'].unique()
      if (pd.isna(unique_source_name)) or (unique_source_name.size == 0):
          print("Destination Center :", i, "-" * 10, "Destination Name :", 'Not
      ↪Found')
      else:
          print("Destination Center :", i, "-" * 10, "Destination Name :",
      ↪unique_destination_name)

```

```

Destination Center : IND342902A1B ----- Destination Name : Not Found

```

```

Destination Center : IND577116AAA ----- Destination Name : Not Found
Destination Center : IND282002AAD ----- Destination Name : Not Found
Destination Center : IND465333A1B ----- Destination Name : Not Found
Destination Center : IND841301AAC ----- Destination Name : Not Found
Destination Center : IND509103AAC ----- Destination Name : Not Found
Destination Center : IND126116AAA ----- Destination Name : Not Found
Destination Center : IND331022A1B ----- Destination Name : Not Found
Destination Center : IND505326AAB ----- Destination Name : Not Found
Destination Center : IND852118A1B ----- Destination Name : Not Found

```

```

[27]: missing_destination_name = df.loc[df['destination_name'].isnull(),
    ↪ 'destination_center'].unique()
missing_destination_name

```

```

[27]: array(['IND342902A1B', 'IND577116AAA', 'IND282002AAD', 'IND465333A1B',
    'IND841301AAC', 'IND505326AAB', 'IND852118A1B', 'IND126116AAA',
    'IND509103AAC', 'IND221005A1A', 'IND250002AAC', 'IND331001A1C',
    'IND122015AAC'], dtype=object)

```

0.1.3 The IDs for which the source name is missing, are all those IDs for destination also missing ?

```

[29]: np.all(df.loc[df['source_name'].isnull(), 'source_center'].
    ↪isin(missing_destination_name))

```

```

[29]: np.False_

```

0.1.4 Treating missing destination names and source names

```

[35]: count = 1
for i in missing_destination_name:
    df.loc[df['destination_center'] == i, 'destination_name'] = df.
    ↪loc[df['destination_center'] == i, 'destination_name'].replace(np.nan,
    ↪f'location_{count}')
    count += 1

```

```

[36]: d = {}
for i in missing_source_name:
    d[i] = df.loc[df['destination_center'] == i, 'destination_name'].unique()
for idx, val in d.items():
    if len(val) == 0:
        d[idx] = [f'location_{count}']
        count += 1
d2 = {}
for idx, val in d.items():
    d2[idx] = val[0]
for i, v in d2.items():

```

```
print(i, v)
```

```
IND342902A1B Destination_1
IND577116AAA Destination_2
IND282002AAD Destination_3
IND465333A1B Destination_4
IND841301AAC Destination_5
IND509103AAC Destination_9
IND126116AAA Destination_8
IND331022A1B location_14
IND505326AAB Destination_6
IND852118A1B Destination_7
```

```
[38]: for i in missing_source_name:
        df.loc[df['source_center'] == i, 'source_name'] = df.
        loc[df['source_center'] == i, 'source_name'].replace(np.nan, d2[i])
```

```
[39]: df.isna().sum()
```

```
[39]: data
trip_creation_time      0
route_schedule_uuid     0
route_type              0
trip_uuid              0
source_center           0
source_name             0
destination_center      0
destination_name        0
od_start_time           0
od_end_time             0
start_scan_to_end_scan  0
actual_distance_to_destination  0
actual_time             0
osrm_time               0
osrm_distance           0
segment_actual_time     0
segment_osrm_time       0
segment_osrm_distance   0
dtype: int64
```

0.1.5 Basic Description of the Data

```
[40]: df.describe()
```

```
[40]:
```

	trip_creation_time	od_start_time \
count	144867	144867
mean	2018-09-22 13:34:23.659819264	2018-09-22 18:02:45.855230720
min	2018-09-12 00:00:16.535741	2018-09-12 00:00:16.535741

25%	2018-09-17 03:20:51.775845888	2018-09-17 08:05:40.886155008
50%	2018-09-22 04:24:27.932764928	2018-09-22 08:53:00.116656128
75%	2018-09-27 17:57:56.350054912	2018-09-27 22:41:50.285857024
max	2018-10-03 23:59:42.701692	2018-10-06 04:27:23.392375
std	NaN	NaN

	od_end_time	start_scan_to_end_scan \
count	144867	144867.000000
mean	2018-09-23 10:04:31.395393024	961.262986
min	2018-09-12 00:50:10.814399	20.000000
25%	2018-09-18 01:48:06.410121984	161.000000
50%	2018-09-23 03:13:03.520212992	449.000000
75%	2018-09-28 12:49:06.054018048	1634.000000
max	2018-10-08 03:00:24.353479	7898.000000
std	NaN	1037.012769

	actual_distance_to_destination	actual_time	osrm_time \
count	144867.000000	144867.000000	144867.000000
mean	234.073380	416.927521	213.868286
min	9.000046	9.000000	6.000000
25%	23.355875	51.000000	27.000000
50%	66.126572	132.000000	64.000000
75%	286.708878	513.000000	257.000000
max	1927.447754	4532.000000	1686.000000
std	344.990021	598.103638	308.011078

	osrm_distance	segment_actual_time	segment_osrm_time \
count	144867.000000	144867.000000	144867.000000
mean	284.771301	36.196110	18.507547
min	9.008200	-244.000000	0.000000
25%	29.914701	20.000000	11.000000
50%	78.525803	29.000000	17.000000
75%	343.193253	40.000000	22.000000
max	2326.199219	3051.000000	1611.000000
std	421.119293	53.571156	14.775960

	segment_osrm_distance
count	144867.000000
mean	22.829018
min	0.000000
25%	12.070100
50%	23.513000
75%	27.813250
max	2191.403809
std	17.860661

```
[41]: df.describe(include = 'object')
```

```
[41]:
```

	route_schedule_uuid \
count	144867
unique	1504
top	thanos::sroute:4029a8a2-6c74-4b7e-a6d8-f9e069f...
freq	1812

	trip_uuid	source_center	source_name \
count	144867	144867	144867
unique	14817	1508	1508
top	trip-153759210483476123	IND000000ACB	Gurgaon_Bilaspur_HB (Haryana)
freq	101	23347	23347

	destination_center	destination_name
count	144867	144867
unique	1481	1481
top	IND000000ACB	Gurgaon_Bilaspur_HB (Haryana)
freq	15192	15192

0.1.6 Merging of rows and aggregation of fields

- Since delivery details of one package are divided into several rows (think of it as connecting flights to reach a particular destination). Now think about how we should treat their fields if we combine these rows? What aggregation would make sense if we merge. What would happen to the numeric fields if we merge the rows.

```
[42]: grouping_1 = ['trip_uuid', 'source_center', 'destination_center']
df1 = df.groupby(by = grouping_1, as_index = False).agg({'data' : 'first',
                                                         'route_type' : 'first',
                                                         'trip_creation_time' : 'first',
                                                         'source_name' : 'first',
                                                         'destination_name' : 'first',
                                                         'od_start_time' : 'first',
                                                         'od_end_time' : 'first',
                                                         'start_scan_to_end_scan' : 'first',
                                                         'actual_distance_to_destination' : 'last',
                                                         'actual_time' : 'last',
                                                         'osrm_time' : 'last',
                                                         'osrm_distance' : 'last',
                                                         'segment_actual_time' : 'sum',
                                                         'segment_osrm_time' : 'sum'})
```

```

        'segment_osrm_distance' :
    ↪ 'sum'})
df1

```

```

[42]:
      trip_uuid  source_center  destination_center  data \
0    trip-153671041653548748  IND209304AAA      IND000000ACB  training
1    trip-153671041653548748  IND462022AAA      IND209304AAA  training
2    trip-153671042288605164  IND561203AAB      IND562101AAA  training
3    trip-153671042288605164  IND572101AAA      IND561203AAB  training
4    trip-153671043369099517  IND000000ACB      IND160002AAC  training
...
26363 trip-153861115439069069  IND628204AAA      IND627657AAA  test
26364 trip-153861115439069069  IND628613AAA      IND627005AAA  test
26365 trip-153861115439069069  IND628801AAA      IND628204AAA  test
26366 trip-153861118270144424  IND583119AAA      IND583101AAA  test
26367 trip-153861118270144424  IND583201AAA      IND583119AAA  test

      route_type      trip_creation_time \
0          FTL 2018-09-12 00:00:16.535741
1          FTL 2018-09-12 00:00:16.535741
2        Carting 2018-09-12 00:00:22.886430
3        Carting 2018-09-12 00:00:22.886430
4          FTL 2018-09-12 00:00:33.691250
...
26363        Carting 2018-10-03 23:59:14.390954
26364        Carting 2018-10-03 23:59:14.390954
26365        Carting 2018-10-03 23:59:14.390954
26366          FTL 2018-10-03 23:59:42.701692
26367          FTL 2018-10-03 23:59:42.701692

      source_name \
0  Kanpur_Central_H_6 (Uttar Pradesh)
1  Bhopal_Trnsport_H (Madhya Pradesh)
2  Doddablpur_ChikaDPP_D (Karnataka)
3  Tumkur_Veersagr_I (Karnataka)
4  Gurgaon_Bilaspur_HB (Haryana)
...
26363 Tirchchnr_Shnmgprn_D (Tamil Nadu)
26364 Peikulam_SriVnktprn_D (Tamil Nadu)
26365 Eral_Busstand_D (Tamil Nadu)
26366 Sandur_WrdN1DPP_D (Karnataka)
26367 Hospet (Karnataka)

      destination_name      od_start_time \
0  Gurgaon_Bilaspur_HB (Haryana) 2018-09-12 16:39:46.858469
1  Kanpur_Central_H_6 (Uttar Pradesh) 2018-09-12 00:00:16.535741
2  Chikblapur_ShntiSgr_D (Karnataka) 2018-09-12 02:03:09.655591

```

3	Doddablpur_ChikaDPP_D (Karnataka)	2018-09-12 00:00:22.886430
4	Chandigarh_Mehmdpur_H (Punjab)	2018-09-14 03:40:17.106733
...	...	...
26363	Thisayanvilai_UdnkdiRD_D (Tamil Nadu)	2018-10-04 02:29:04.272194
26364	Tirunelveli_VdkkuSrt_I (Tamil Nadu)	2018-10-04 04:16:39.894872
26365	Tirchchndr_Shnmgprn_D (Tamil Nadu)	2018-10-04 01:44:53.808000
26366	Bellary_Dc (Karnataka)	2018-10-04 03:58:40.726547
26367	Sandur_WrdN1DPP_D (Karnataka)	2018-10-04 02:51:44.712656

	od_end_time	start_scan_to_end_scan \
0	2018-09-13 13:40:23.123744	1260.0
1	2018-09-12 16:39:46.858469	999.0
2	2018-09-12 03:01:59.598855	58.0
3	2018-09-12 02:03:09.655591	122.0
4	2018-09-14 17:34:55.442454	834.0
...	...	...
26363	2018-10-04 03:31:11.183797	62.0
26364	2018-10-04 05:47:45.162682	91.0
26365	2018-10-04 02:29:04.272194	44.0
26366	2018-10-04 08:46:09.166940	287.0
26367	2018-10-04 03:58:40.726547	66.0

	actual_distance_to_destination	actual_time	osrm_time	osrm_distance \
0	383.759155	732.0	329.0	446.549591
1	440.973694	830.0	388.0	544.802673
2	24.644020	47.0	26.0	28.199400
3	48.542889	96.0	42.0	56.911598
4	237.439606	611.0	212.0	281.210907
...	...	...	...	...
26363	33.627182	51.0	41.0	42.521301
26364	33.673836	90.0	48.0	40.608002
26365	12.661944	30.0	14.0	16.018499
26366	40.546738	233.0	42.0	52.530300
26367	25.534794	42.0	26.0	28.048401

	segment_actual_time	segment_osrm_time	segment_osrm_distance
0	728.0	534.0	670.620483
1	820.0	474.0	649.852783
2	46.0	26.0	28.199501
3	95.0	39.0	55.989899
4	608.0	231.0	317.740784
...	...	...	...
26363	49.0	42.0	42.143101
26364	89.0	77.0	78.586899
26365	29.0	14.0	16.018400
26366	233.0	42.0	52.530300
26367	41.0	25.0	28.048401

[26368 rows x 18 columns]

0.1.7 Calculate the time taken between `od_start_time` and `od_end_time` and keep it as a feature. Drop the original columns, if required

```
[43]: df1['od_total_time'] = df1['od_end_time'] - df1['od_start_time']
df1.drop(columns = ['od_end_time', 'od_start_time'], inplace = True)
df1['od_total_time'] = df1['od_total_time'].apply(lambda x : round(x.
    ↳total_seconds() / 60.0, 2))
df1['od_total_time'].head()
```

```
[43]: 0    1260.60
1     999.51
2      58.83
3     122.78
4     834.64
Name: od_total_time, dtype: float64
```

```
[44]: df2 = df1.groupby(by = 'trip_uuid', as_index = False).agg({'source_center' :↳
    ↳'first',
                                                                    'destination_center'↳
    ↳: 'last',
                                                                    'data' : 'first',
                                                                    'route_type' :↳
    ↳'first',
                                                                    'trip_creation_time'↳
    ↳: 'first',
                                                                    'source_name' :↳
    ↳'first',
                                                                    'destination_name' :↳
    ↳'last',
                                                                    'od_total_time' :↳
    ↳'sum',
                                                                    ↳
    ↳'start_scan_to_end_scan' : 'sum',
                                                                    ↳
    ↳'actual_distance_to_destination' : 'sum',
                                                                    'actual_time' :↳
    ↳'sum',
                                                                    'osrm_time' : 'sum',
                                                                    'osrm_distance' :↳
    ↳'sum',
                                                                    ↳
    ↳'segment_actual_time' : 'sum',
```

```

        'segment_osrm_time' :
↪ 'sum',
        ↪
↪ 'segment_osrm_distance' : 'sum'})
df2

```

```

[44]:
      trip_uuid source_center destination_center data \
0      trip-153671041653548748  IND209304AAA  IND209304AAA  training
1      trip-153671042288605164  IND561203AAB  IND561203AAB  training
2      trip-153671043369099517  IND000000ACB  IND000000ACB  training
3      trip-153671046011330457  IND400072AAB  IND401104AAA  training
4      trip-153671052974046625  IND583101AAA  IND583119AAA  training
...
14812  trip-153861095625827784  IND160002AAC  IND160002AAC  test
14813  trip-153861104386292051  IND121004AAB  IND121004AAA  test
14814  trip-153861106442901555  IND208006AAA  IND208006AAA  test
14815  trip-153861115439069069  IND627005AAA  IND628204AAA  test
14816  trip-153861118270144424  IND583119AAA  IND583119AAA  test

      route_type      trip_creation_time \
0      FTL 2018-09-12 00:00:16.535741
1      Carting 2018-09-12 00:00:22.886430
2      FTL 2018-09-12 00:00:33.691250
3      Carting 2018-09-12 00:01:00.113710
4      FTL 2018-09-12 00:02:09.740725
...
14812  Carting 2018-10-03 23:55:56.258533
14813  Carting 2018-10-03 23:57:23.863155
14814  Carting 2018-10-03 23:57:44.429324
14815  Carting 2018-10-03 23:59:14.390954
14816  FTL 2018-10-03 23:59:42.701692

      source_name \
0      Kanpur_Central_H_6 (Uttar Pradesh)
1      Doddablpur_ChikaDPP_D (Karnataka)
2      Gurgaon_Bilaspur_HB (Haryana)
3      Mumbai Hub (Maharashtra)
4      Bellary_Dc (Karnataka)
...
14812  Chandigarh_Mehmdpur_H (Punjab)
14813  FBD_Balabhgarh_DPC (Haryana)
14814  Kanpur_GovndNgr_DC (Uttar Pradesh)
14815  Tirunelveli_VdkkuSrt_I (Tamil Nadu)
14816  Sandur_WrdN1DPP_D (Karnataka)

      destination_name  od_total_time \
0      Kanpur_Central_H_6 (Uttar Pradesh)  2260.11

```

1	Doddablpur_ChikaDPP_D (Karnataka)	181.61
2	Gurgaon_Bilaspur_HB (Haryana)	3934.36
3	Mumbai_MiraRd_IP (Maharashtra)	100.49
4	Sandur_WrdN1DPP_D (Karnataka)	718.34
...	...	...
14812	Chandigarh_Mehmdpur_H (Punjab)	258.03
14813	Faridabad_Blbgarh_DC (Haryana)	60.59
14814	Kanpur_GovndNgr_DC (Uttar Pradesh)	422.12
14815	Tirchchndr_Shnmgprn_D (Tamil Nadu)	348.52
14816	Sandur_WrdN1DPP_D (Karnataka)	354.40

	start_scan_to_end_scan	actual_distance_to_destination	actual_time	\
0	2259.0	824.732849	1562.0	
1	180.0	73.186905	143.0	
2	3933.0	1927.404297	3347.0	
3	100.0	17.175274	59.0	
4	717.0	127.448502	341.0	
...	...	...	...	
14812	257.0	57.762333	83.0	
14813	60.0	15.513784	21.0	
14814	421.0	38.684837	282.0	
14815	347.0	134.723831	264.0	
14816	353.0	66.081528	275.0	

	osrm_time	osrm_distance	segment_actual_time	segment_osrm_time	\
0	717.0	991.352295	1548.0	1008.0	
1	68.0	85.111000	141.0	65.0	
2	1740.0	2354.066650	3308.0	1941.0	
3	15.0	19.680000	59.0	16.0	
4	117.0	146.791794	340.0	115.0	
...	...	...	...	...	
14812	62.0	73.462997	82.0	62.0	
14813	12.0	16.088200	21.0	11.0	
14814	48.0	58.903702	281.0	88.0	
14815	179.0	171.110306	258.0	221.0	
14816	68.0	80.578705	274.0	67.0	

	segment_osrm_distance
0	1320.473267
1	84.189400
2	2545.267822
3	19.876600
4	146.791901
...	...
14812	64.855103
14813	16.088299
14814	104.886597

```
14815          223.532394
14816          80.578705
```

```
[14817 rows x 17 columns]
```

0.1.8 2. Build some features to prepare the data for actual analysis. Extract features from the below fields:

0.1.9 Source Name: Split and extract features out of destination. City-place-code (State)

```
[45]: def location_name_to_state(x):
      l = x.split('(')
      if len(l) == 1:
          return l[0]
      else:
          return l[1].replace(')', '')
```

```
[46]: def location_name_to_city(x):
      if 'location' in x:
          return 'unknown_city'
      else:
          l = x.split()[0].split('_')
          if 'CCU' in x:
              return 'Kolkata'
          elif 'MAA' in x.upper():
              return 'Chennai'
          elif ('HBR' in x.upper()) or ('BLR' in x.upper()):
              return 'Bengaluru'
          elif 'FBD' in x.upper():
              return 'Faridabad'
          elif 'BOM' in x.upper():
              return 'Mumbai'
          elif 'DEL' in x.upper():
              return 'Delhi'
          elif 'OK' in x.upper():
              return 'Delhi'
          elif 'GZB' in x.upper():
              return 'Ghaziabad'
          elif 'GGN' in x.upper():
              return 'Gurgaon'
          elif 'AMD' in x.upper():
              return 'Ahmedabad'
          elif 'CJB' in x.upper():
              return 'Coimbatore'
          elif 'HYD' in x.upper():
              return 'Hyderabad'
```

```
return l[0]
```

```
[47]: def location_name_to_place(x):  
    if 'location' in x:  
        return x  
    elif 'HBR' in x:  
        return 'HBR Layout PC'  
    else:  
        l = x.split()[0].split('_', 1)  
        if len(l) == 1:  
            return 'unknown_place'  
        else:  
            return l[1]
```

```
[48]: df2['source_state'] = df2['source_name'].apply(location_name_to_state)  
df2['source_state'].unique()
```

```
[48]: array(['Uttar Pradesh', 'Karnataka', 'Haryana', 'Maharashtra',  
        'Tamil Nadu', 'Gujarat', 'Delhi', 'Telangana', 'Rajasthan',  
        'Assam', 'Madhya Pradesh', 'West Bengal', 'Andhra Pradesh',  
        'Punjab', 'Chandigarh', 'Goa', 'Jharkhand', 'Pondicherry',  
        'Orissa', 'Uttarakhand', 'Himachal Pradesh', 'Kerala',  
        'Arunachal Pradesh', 'Bihar', 'Chhattisgarh',  
        'Dadra and Nagar Haveli', 'Jammu & Kashmir', 'Mizoram', 'Nagaland',  
        'Destination_9', 'Destination_3', 'Destination_2', 'location_14',  
        'Destination_7'], dtype=object)
```

```
[49]: df2['source_city'] = df2['source_name'].apply(location_name_to_city)  
print('No of source cities :', df2['source_city'].nunique())  
df2['source_city'].unique()[:100]
```

No of source cities : 691

```
[49]: array(['Kanpur', 'Doddablpur', 'Gurgaon', 'Mumbai', 'Bellary', 'Chennai',  
        'Bengaluru', 'Surat', 'Delhi', 'Pune', 'Faridabad', 'Shirala',  
        'Hyderabad', 'Thirumalagiri', 'Gulbarga', 'Jaipur', 'Allahabad',  
        'Guwahati', 'Narsinghpur', 'Shrirampur', 'Madakasira', 'Sonari',  
        'Dindigul', 'Jalandhar', 'Chandigarh', 'Deoli', 'Pandharpur',  
        'Kolkata', 'Bhandara', 'Kurnool', 'Bhiwandi', 'Bhatinda',  
        'RoopNagar', 'Bantwal', 'Lalru', 'Kadi', 'Shahdol', 'Gangakher',  
        'Durgapur', 'Vapi', 'Jamjodhpur', 'Jetpur', 'Mehsana', 'Jabalpur',  
        'Junagadh', 'Gundlupet', 'Mysore', 'Goa', 'Bhopal', 'Sonipat',  
        'Himmatnagar', 'Jamshedpur', 'Pondicherry', 'Anand', 'Udgir',  
        'Nadiad', 'Villupuram', 'Purulia', 'Bhubaneshwar', 'Bamangola',  
        'Tiruppattur', 'Kotdwara', 'Medak', 'Bangalore', 'Dhrangadhra',  
        'Hospet', 'Ghumarwin', 'Agra', 'Sitapur', 'Canacona', 'Bilimora',  
        'SultnBthry', 'Lucknow', 'Vellore', 'Bhuj', 'Dinhata',  
        'Margherita', 'Boisar', 'Vizag', 'Tezpur', 'Koduru', 'Tirupati',
```

```
'Pen', 'Ahmedabad', 'Faizabad', 'Gandhinagar', 'Anantapur',
'Betul', 'Panskura', 'Rasipuram', 'Sankari', 'Jorhat', 'PNQ',
'Srikakulam', 'Dehradun', 'Jassur', 'Sawantwadi', 'Shajapur',
'Ludhiana', 'GreaterThane'], dtype=object)
```

```
[50]: df2['source_place'] = df2['source_name'].apply(location_name_to_place)
df2['source_place'].unique()[:100]
```

```
[50]: array(['Central_H_6', 'ChikaDPP_D', 'Bilaspur_HB', 'unknown_place', 'Dc',
'Poonamallee', 'Chrompet_DPC', 'HBR Layout PC', 'Central_D_12',
'Lajpat_IP', 'North_D_3', 'Balabharhgarh_DPC', 'Central_DPP_3',
'Shamshbd_H', 'Xroad_D', 'Nehrugn_I', 'Central_I_7',
'Central_H_1', 'Nangli_IP', 'North', 'KndliDPP_D', 'Central_D_9',
'DavkharRd_D', 'Bandel_D', 'RTCStand_D', 'Central_DPP_1',
'KGairprt_HB', 'North_D_2', 'Central_D_1', 'DC', 'Mthurard_L',
'Mullanpr_DC', 'Central_DPP_2', 'RajCmplx_D', 'Beliaghata_DPC',
'RjnaiDPP_D', 'AbbasNgr_I', 'Mankoli_HB', 'DPC', 'Airport_H',
'Hub', 'Gateway_HB', 'Tathawde_H', 'ChotiHvl_DC', 'Trmltpl_D',
'OnkarDPP_D', 'Mehmdpur_H', 'KaranNGR_D', 'Sohagpur_D',
'Chrompet_L', 'Busstand_D', 'Central_I_1', 'IndEstat_I', 'Court_D',
'Panchot_IP', 'Adhartal_IP', 'DumDum_DPC', 'Bomsndra_HB',
'Swamylyt_D', 'Yadvigiri_IP', 'Old', 'Kundli_H', 'Central_I_3',
'Vasanthm_I', 'Poonamallee_HB', 'VUNagar_DC', 'NlgaonRd_D',
'Bnnrgha_L', 'Thirumtr_IP', 'GariDPP_D', 'Jogshwri_I',
'Koilstrt_D', 'CotnGren_M', 'Nzbadrd_D', 'Dwaraka_D', 'Nelmnnga_H',
'NvygRDPP_D', 'Gndhichk_D', 'Central_D_3', 'Chowk_D', 'CharRsta_D',
'Kollgpra_D', 'Peenya_IP', 'GndhiNgr_IP', 'Sanpada_I',
'WrdN4DPP_D', 'Sakinaka_RP', 'CivilHPL_D', 'OstwlEmp_D',
'Gajuwaka', 'Mhbhirab_D', 'MGRoad_D', 'Balajicly_I', 'BljiMrkt_D',
'Dankuni_HB', 'Trnsport_H', 'Rakhial', 'Memnagar', 'East_I_21',
'Mithakal_D'], dtype=object)
```

0.1.10 Destination Name: Split and extract features out of destination. City-place-code (State)

```
[51]: df2['destination_state'] = df2['destination_name'].apply(location_name_to_state)
df2['destination_state'].head(10)
```

```
[51]: 0    Uttar Pradesh
1      Karnataka
2      Haryana
3    Maharashtra
4      Karnataka
5    Tamil Nadu
6    Tamil Nadu
7      Karnataka
8      Gujarat
```

```
9          Delhi
Name: destination_state, dtype: object
```

```
[52]: df2['destination_city'] = df2['destination_name'].apply(location_name_to_city)
df2['destination_city'].head()
```

```
[52]: 0      Kanpur
1      Doddablpur
2      Gurgaon
3      Mumbai
4      Sandur
Name: destination_city, dtype: object
```

```
[53]: df2['destination_place'] = df2['destination_name'].apply(location_name_to_place)
df2['destination_place'].head()
```

```
[53]: 0      Central_H_6
1      ChikaDPP_D
2      Bilaspur_HB
3      MiraRd_IP
4      WrdN1DPP_D
Name: destination_place, dtype: object
```

0.1.11 Trip_creation_time: Extract features like month, year and day etc

```
[54]: df2['trip_creation_date'] = pd.to_datetime(df2['trip_creation_time'].dt.date)
df2['trip_creation_date'].head()
```

```
[54]: 0      2018-09-12
1      2018-09-12
2      2018-09-12
3      2018-09-12
4      2018-09-12
Name: trip_creation_date, dtype: datetime64[ns]
```

```
[55]: df2['trip_creation_day'] = df2['trip_creation_time'].dt.day
df2['trip_creation_day'] = df2['trip_creation_day'].astype('int8')
df2['trip_creation_day'].head()
```

```
[55]: 0      12
1      12
2      12
3      12
4      12
Name: trip_creation_day, dtype: int8
```

```
[56]: df2['trip_creation_month'] = df2['trip_creation_time'].dt.month
df2['trip_creation_month'] = df2['trip_creation_month'].astype('int8')
df2['trip_creation_month'].head()
```

```
[56]: 0    9
      1    9
      2    9
      3    9
      4    9
      Name: trip_creation_month, dtype: int8
```

```
[57]: df2['trip_creation_year'] = df2['trip_creation_time'].dt.year
df2['trip_creation_year'] = df2['trip_creation_year'].astype('int16')
df2['trip_creation_year'].head()
```

```
[57]: 0    2018
      1    2018
      2    2018
      3    2018
      4    2018
      Name: trip_creation_year, dtype: int16
```

```
[58]: df2['trip_creation_week'] = df2['trip_creation_time'].dt.isocalendar().week
df2['trip_creation_week'] = df2['trip_creation_week'].astype('int8')
df2['trip_creation_week'].head()
```

```
[58]: 0    37
      1    37
      2    37
      3    37
      4    37
      Name: trip_creation_week, dtype: int8
```

```
[59]: df2['trip_creation_hour'] = df2['trip_creation_time'].dt.hour
df2['trip_creation_hour'] = df2['trip_creation_hour'].astype('int8')
df2['trip_creation_hour'].head()
```

```
[59]: 0    0
      1    0
      2    0
      3    0
      4    0
      Name: trip_creation_hour, dtype: int8
```

Finding the structure of data after data cleaning

```
[60]: df2.shape
```

[60]: (14817, 29)

[61]: df2.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14817 entries, 0 to 14816
Data columns (total 29 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   trip_uuid                            14817 non-null  object
1   source_center                        14817 non-null  object
2   destination_center                  14817 non-null  object
3   data                                14817 non-null  category
4   route_type                          14817 non-null  category
5   trip_creation_time                  14817 non-null  datetime64[ns]
6   source_name                         14817 non-null  object
7   destination_name                   14817 non-null  object
8   od_total_time                      14817 non-null  float64
9   start_scan_to_end_scan             14817 non-null  float64
10  actual_distance_to_destination      14817 non-null  float32
11  actual_time                        14817 non-null  float32
12  osrm_time                         14817 non-null  float32
13  osrm_distance                     14817 non-null  float32
14  segment_actual_time                14817 non-null  float32
15  segment_osrm_time                 14817 non-null  float32
16  segment_osrm_distance              14817 non-null  float32
17  source_state                      14817 non-null  object
18  source_city                       14817 non-null  object
19  source_place                      14817 non-null  object
20  destination_state                 14817 non-null  object
21  destination_city                  14817 non-null  object
22  destination_place                 14817 non-null  object
23  trip_creation_date                 14817 non-null  datetime64[ns]
24  trip_creation_day                  14817 non-null  int8
25  trip_creation_month                14817 non-null  int8
26  trip_creation_year                 14817 non-null  int16
27  trip_creation_week                 14817 non-null  int8
28  trip_creation_hour                 14817 non-null  int8
dtypes: category(2), datetime64[ns](2), float32(7), float64(2), int16(1),
int8(4), object(11)
memory usage: 2.2+ MB
```

[62]: df2.describe().T

```
[62]:
```

	count	mean \
trip_creation_time	14817	2018-09-22 12:44:19.555167744
od_total_time	14817.0	531.69763
start_scan_to_end_scan	14817.0	530.810016

actual_distance_to_destination	14817.0	164.477829
actual_time	14817.0	357.143768
osrm_time	14817.0	161.384018
osrm_distance	14817.0	204.344711
segment_actual_time	14817.0	353.892273
segment_osrm_time	14817.0	180.949783
segment_osrm_distance	14817.0	223.201157
trip_creation_date	14817	2018-09-21 23:46:58.627252736
trip_creation_day	14817.0	18.37079
trip_creation_month	14817.0	9.120672
trip_creation_year	14817.0	2018.0
trip_creation_week	14817.0	38.295944
trip_creation_hour	14817.0	12.449821

		min \
trip_creation_time	2018-09-12 00:00:16.535741	
od_total_time		23.46
start_scan_to_end_scan		23.0
actual_distance_to_destination		9.002461
actual_time		9.0
osrm_time		6.0
osrm_distance		9.0729
segment_actual_time		9.0
segment_osrm_time		6.0
segment_osrm_distance		9.0729
trip_creation_date	2018-09-12 00:00:00	
trip_creation_day		1.0
trip_creation_month		9.0
trip_creation_year		2018.0
trip_creation_week		37.0
trip_creation_hour		0.0

		25% \
trip_creation_time	2018-09-17 02:51:25.129125888	
od_total_time		149.93
start_scan_to_end_scan		149.0
actual_distance_to_destination		22.837238
actual_time		67.0
osrm_time		29.0
osrm_distance		30.819201
segment_actual_time		66.0
segment_osrm_time		31.0
segment_osrm_distance		32.654499
trip_creation_date	2018-09-17 00:00:00	
trip_creation_day		14.0
trip_creation_month		9.0
trip_creation_year		2018.0

trip_creation_week		38.0
trip_creation_hour		4.0
		50% \
trip_creation_time	2018-09-22 04:02:35.066945024	
od_total_time		280.77
start_scan_to_end_scan		280.0
actual_distance_to_destination		48.474072
actual_time		149.0
osrm_time		60.0
osrm_distance		65.618805
segment_actual_time		147.0
segment_osrm_time		65.0
segment_osrm_distance		70.154404
trip_creation_date	2018-09-22 00:00:00	
trip_creation_day		19.0
trip_creation_month		9.0
trip_creation_year		2018.0
trip_creation_week		38.0
trip_creation_hour		14.0

		75% \
trip_creation_time	2018-09-27 19:37:41.898427904	
od_total_time		638.2
start_scan_to_end_scan		637.0
actual_distance_to_destination		164.583206
actual_time		370.0
osrm_time		168.0
osrm_distance		208.475006
segment_actual_time		367.0
segment_osrm_time		185.0
segment_osrm_distance		218.802399
trip_creation_date	2018-09-27 00:00:00	
trip_creation_day		25.0
trip_creation_month		9.0
trip_creation_year		2018.0
trip_creation_week		39.0
trip_creation_hour		20.0

		max	std
trip_creation_time	2018-10-03 23:59:42.701692		NaN
od_total_time		7898.55	658.868223
start_scan_to_end_scan		7898.0	658.705957
actual_distance_to_destination		2186.531738	305.388153
actual_time		6265.0	561.396179
osrm_time		2032.0	271.360992
osrm_distance		2840.081055	370.395569

segment_actual_time	6230.0	556.247986
segment_osrm_time	2564.0	314.542053
segment_osrm_distance	3523.632324	416.628387
trip_creation_date	2018-10-03 00:00:00	NaN
trip_creation_day	30.0	7.893275
trip_creation_month	10.0	0.325757
trip_creation_year	2018.0	0.0
trip_creation_week	40.0	0.967872
trip_creation_hour	23.0	7.986553

```
[63]: df2.describe(include = object).T
```

```
[63]:
```

	count	unique	top	freq
trip_uuid	14817	14817	trip-153861118270144424	1
source_center	14817	938	IND0000000ACB	1063
destination_center	14817	1042	IND0000000ACB	821
source_name	14817	938	Gurgaon_Bilaspur_HB (Haryana)	1063
destination_name	14817	1042	Gurgaon_Bilaspur_HB (Haryana)	821
source_state	14817	34	Maharashtra	2714
source_city	14817	691	Mumbai	1442
source_place	14817	761	Bilaspur_HB	1063
destination_state	14817	39	Maharashtra	2561
destination_city	14817	806	Mumbai	1548
destination_place	14817	850	Bilaspur_HB	821

I am intrested to know how many trips are created on the hourly basis

```
[64]: df2['trip_creation_hour'].unique()
```

```
[64]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
        17, 18, 19, 20, 21, 22, 23], dtype=int8)
```

```
[65]: df_hour = df2.groupby(by = 'trip_creation_hour')['trip_uuid'].count().
        ↪to_frame().reset_index()
df_hour.head()
```

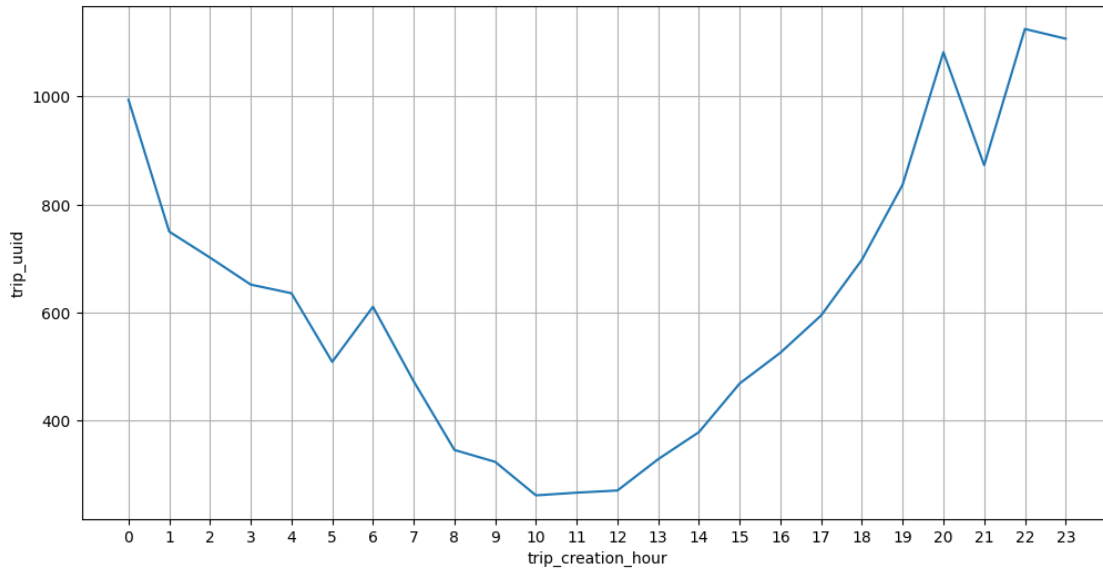
```
[65]:
```

	trip_creation_hour	trip_uuid
0	0	994
1	1	750
2	2	702
3	3	652
4	4	636

```
[66]: plt.figure(figsize = (12, 6))
sns.lineplot(data = df_hour,
              x = df_hour['trip_creation_hour'],
              y = df_hour['trip_uuid'],
              markers = '*')
```

```
plt.xticks(np.arange(0,24))
plt.grid('both')
plt.plot()
```

[66]: []



- It can be inferred from the above plot that the number of trips start increasing after the noon, becomes maximum at 10 P.M and then start decreasing.

I am intrested to know how many trips are created for different days of the month

```
[67]: df2['trip_creation_day'].unique()
```

```
[67]: array([12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28,
          29, 30,  1,  2,  3], dtype=int8)
```

```
[68]: df_day = df2.groupby(by = 'trip_creation_day')['trip_uuid'].count().to_frame().
      ↪reset_index()
      df_day.head()
```

```
[68]:   trip_creation_day  trip_uuid
0                1         605
1                2         552
2                3         631
3               12         747
4               13         750
```

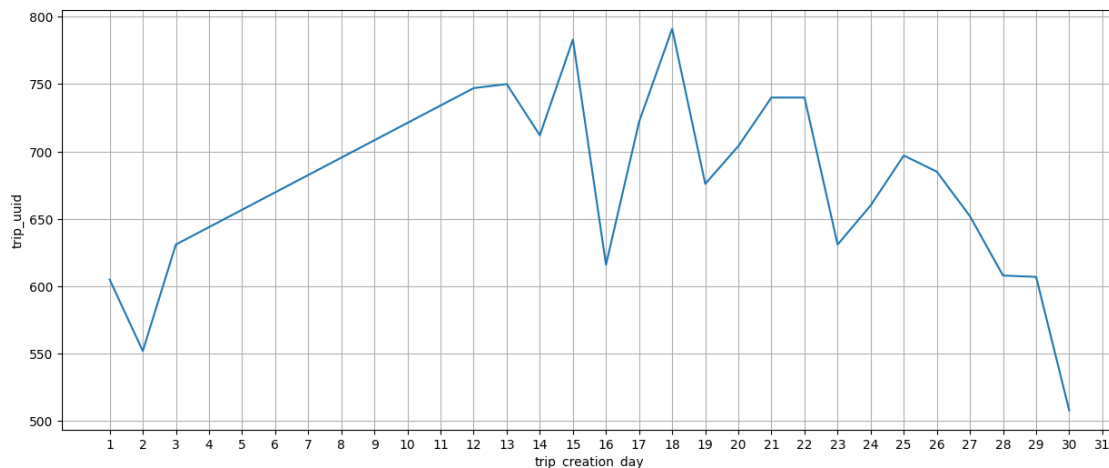
```
[69]: plt.figure(figsize = (15, 6))
      sns.lineplot(data = df_day,
```

```

x = df_day['trip_creation_day'],
y = df_day['trip_uuid'],
markers = 'o')
plt.xticks(np.arange(1, 32))
plt.grid('both')
plt.plot()

```

[69]: []



- It can be inferred from the above plot that most of the trips are created in the mid of the month.
- That means customers usually make more orders in the mid of the month.

I am intrested to know how many trips are created for different weeks

```
[70]: df2['trip_creation_week'].unique()
```

```
[70]: array([37, 38, 39, 40], dtype=int8)
```

```
[71]: df_week = df2.groupby(by = 'trip_creation_week')['trip_uuid'].count().
      ↪to_frame().reset_index()
df_week.head()
```

```
[71]:
```

	trip_creation_week	trip_uuid
0	37	3608
1	38	5004
2	39	4417
3	40	1788

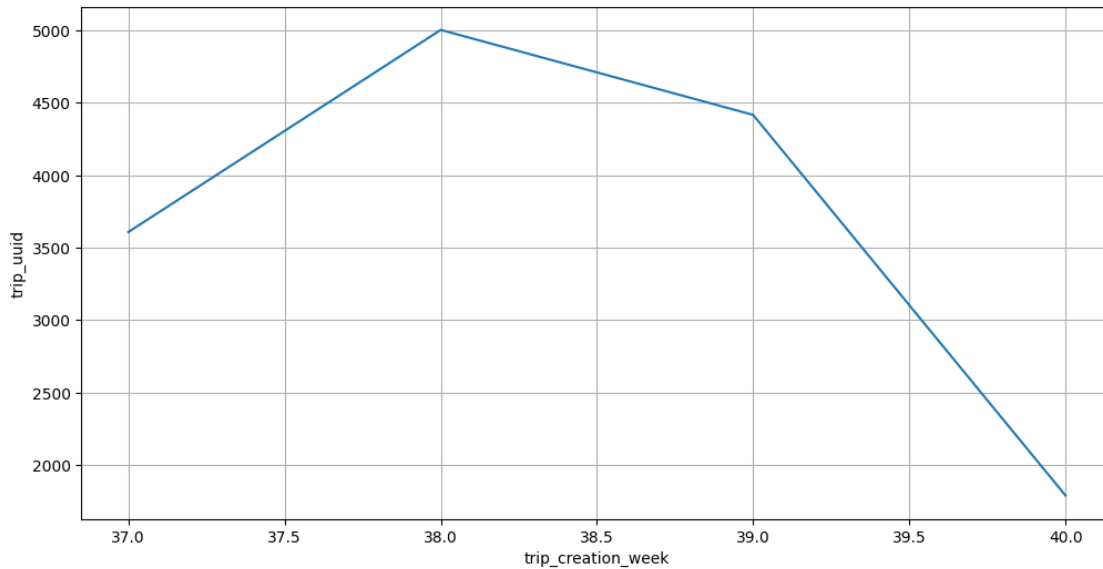
```
[72]: plt.figure(figsize = (12, 6))
sns.lineplot(data = df_week,
             x = df_week['trip_creation_week'],
```

```

        y = df_week['trip_uuid'],
        markers = 'o')
plt.grid('both')
plt.plot()

```

[72]: []



- It can be inferred from the above plot that most of the trips are created in the 38th week.

I am intrested to know how many trips are created in the given two months

```

[73]: df_month = df2.groupby(by = 'trip_creation_month')['trip_uuid'].count().
        ↪to_frame().reset_index()
df_month['perc'] = np.round(df_month['trip_uuid'] * 100/ df_month['trip_uuid'].
        ↪sum(), 2)
df_month.head()

```

```

[73]:   trip_creation_month  trip_uuid  perc
0                9        13029  87.93
1               10         1788  12.07

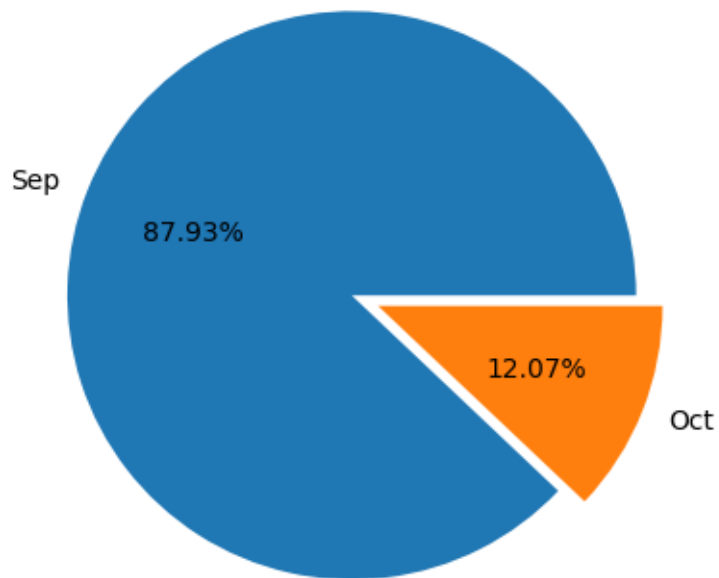
```

```

[74]: plt.pie(x = df_month['trip_uuid'],
        labels = ['Sep', 'Oct'],
        explode = [0, 0.1],
        autopct = '%.2f%%')
plt.plot()

```

[74]: []



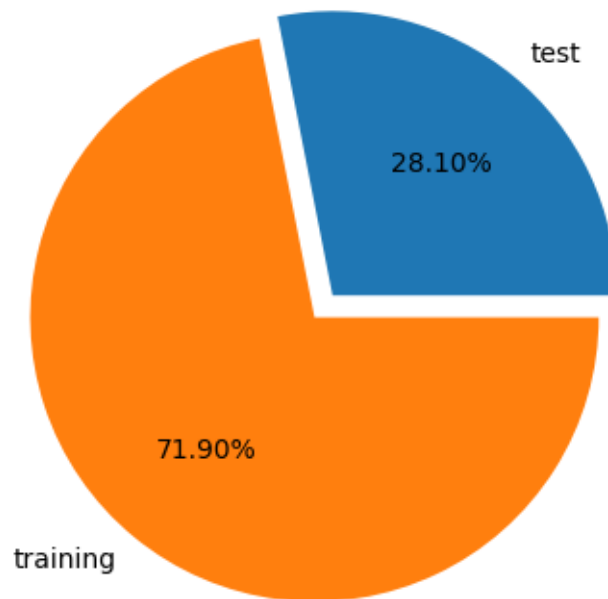
I am interested to know the distribution of trip data for the orders

```
[75]: df_data = df2.groupby(by = 'data')['trip_uuid'].count().to_frame().reset_index()
df_data['perc'] = np.round(df_data['trip_uuid'] * 100/ df_data['trip_uuid'].
    ↳sum(), 2)
df_data.head()
```

```
[75]:      data  trip_uuid  perc
0    test        4163   28.1
1  training       10654   71.9
```

```
[76]: plt.pie(x = df_data['trip_uuid'],
            labels = df_data['data'],
            explode = [0, 0.1],
            autopct = '%.2f%%')
plt.plot()
```

```
[76]: []
```



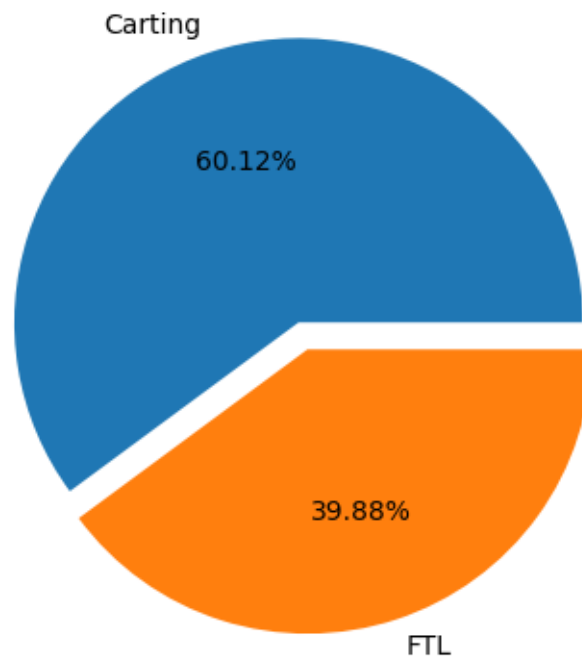
I am interested to know the distribution of route types for the orders

```
[77]: df_route = df2.groupby(by = 'route_type')['trip_uuid'].count().to_frame().
      ↪reset_index()
      df_route['perc'] = np.round(df_route['trip_uuid'] * 100/ df_route['trip_uuid'].
      ↪sum(), 2)
      df_route.head()
```

```
[77]:  route_type  trip_uuid  perc
0    Carting      8908  60.12
1      FTL       5909  39.88
```

```
[78]: plt.pie(x = df_route['trip_uuid'],
              labels = ['Carting', 'FTL'],
              explode = [0, 0.1],
              autopct = '%.2f%%')
plt.plot()
```

```
[78]: []
```



I am interested to know what is the distribution of number of trips created from different states

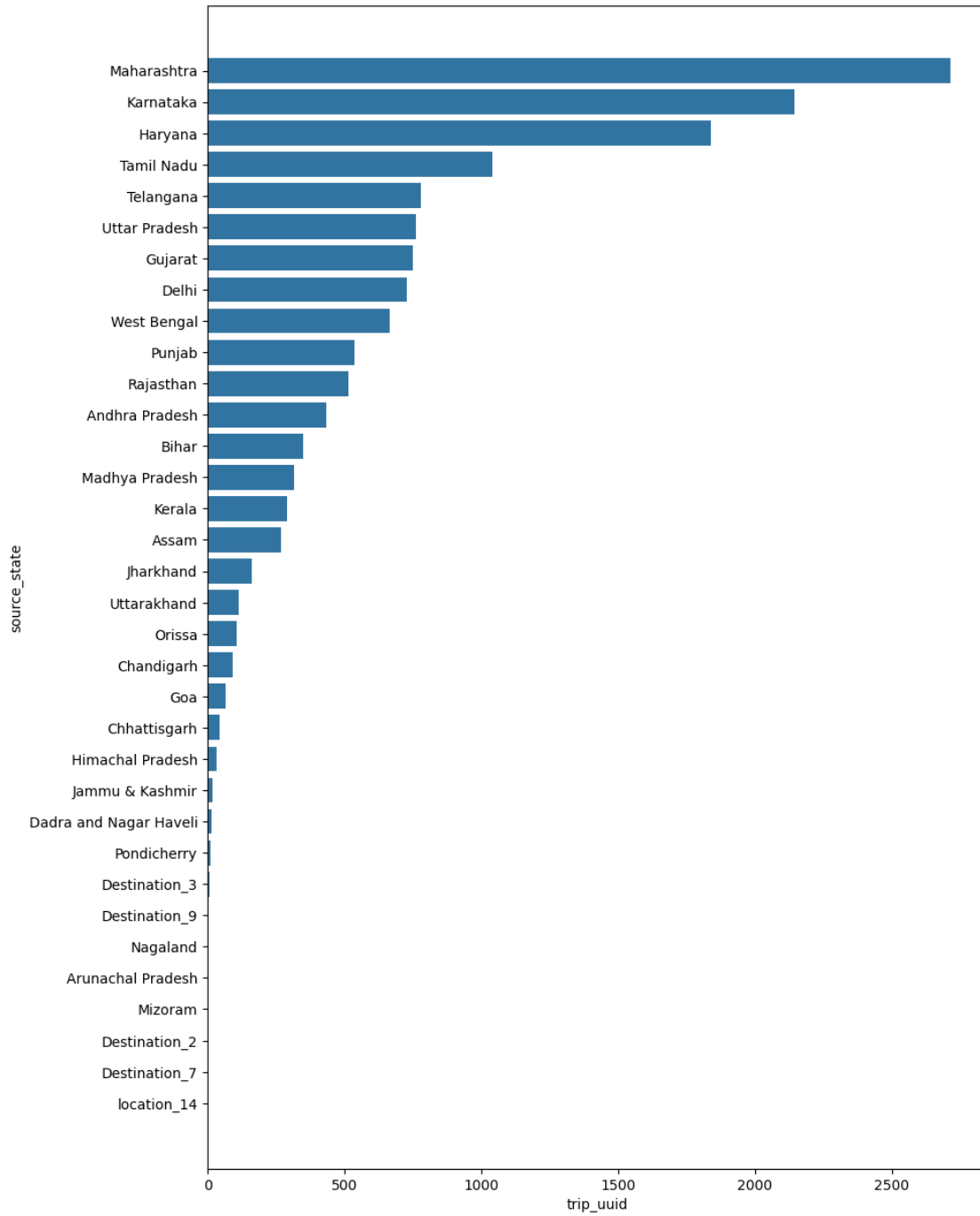
```
[79]: df_source_state = df2.groupby(by = 'source_state')['trip_uuid'].count().
      ↪to_frame().reset_index()
df_source_state['perc'] = np.round(df_source_state['trip_uuid'] * 100/
      ↪df_source_state['trip_uuid'].sum(), 2)
df_source_state = df_source_state.sort_values(by = 'trip_uuid', ascending =
      ↪False)
df_source_state.head()
```

```
[79]:
```

	source_state	trip_uuid	perc
21	Maharashtra	2714	18.32
18	Karnataka	2143	14.46
14	Haryana	1838	12.40
28	Tamil Nadu	1039	7.01
29	Telangana	781	5.27

```
[80]: plt.figure(figsize = (10, 15))
      sns.barplot(data = df_source_state,
                  x = df_source_state['trip_uuid'],
                  y = df_source_state['source_state'])
      plt.plot()
```

[80]: []



- It can be seen in the above plot that maximum trips originated from Maharashtra state followed by Karnataka and Haryana. That means that the seller base is strong in these states

I am interested to know top 30 cities based on the number of trips created from different cities

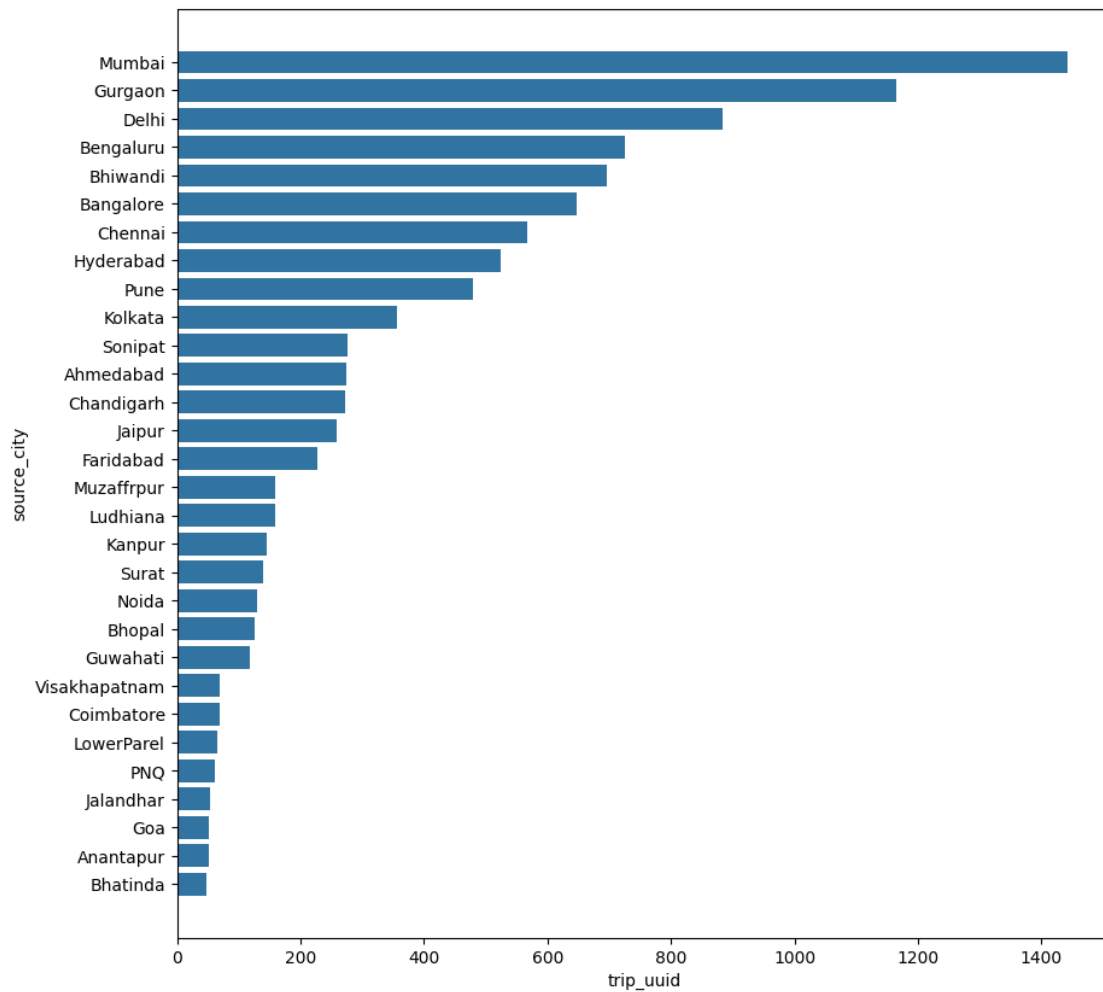
```
[81]: df_source_city = df2.groupby(by = 'source_city')['trip_uuid'].count().
      ↪to_frame().reset_index()
df_source_city['perc'] = np.round(df_source_city['trip_uuid'] * 100/
      ↪df_source_city['trip_uuid'].sum(), 2)
df_source_city = df_source_city.sort_values(by = 'trip_uuid', ascending =
      ↪False)[:30]
df_source_city
```

```
[81]:
```

	source_city	trip_uuid	perc
440	Mumbai	1442	9.73
238	Gurgaon	1165	7.86
169	Delhi	883	5.96
79	Bengaluru	726	4.90
100	Bhiwandi	697	4.70
58	Bangalore	648	4.37
136	Chennai	568	3.83
265	Hyderabad	524	3.54
517	Pune	480	3.24
358	Kolkata	356	2.40
611	Sonipat	276	1.86
2	Ahmedabad	274	1.85
133	Chandigarh	273	1.84
271	Jaipur	259	1.75
202	Faridabad	227	1.53
448	Muzaffarpur	159	1.07
383	Ludhiana	158	1.07
321	Kanpur	145	0.98
622	Surat	140	0.94
474	Noida	129	0.87
102	Bhopal	125	0.84
241	Guwahati	118	0.80
680	Visakhapatnam	69	0.47
154	Coimbatore	69	0.47
381	LowerParel	65	0.44
478	PNQ	62	0.42
274	Jalandhar	54	0.36
221	Goa	52	0.35
25	Anantapur	51	0.34
93	Bhatinda	47	0.32

```
[82]: plt.figure(figsize = (10, 10))
sns.barplot(data = df_source_city,
            x = df_source_city['trip_uuid'],
            y = df_source_city['source_city'])
plt.plot()
```

[82]: []



- It can be seen in the above plot that maximum trips originated from Mumbai city followed by Gurgaon Delhi, Bengaluru and Bhiwandi. That means that the seller base is strong in these cities.

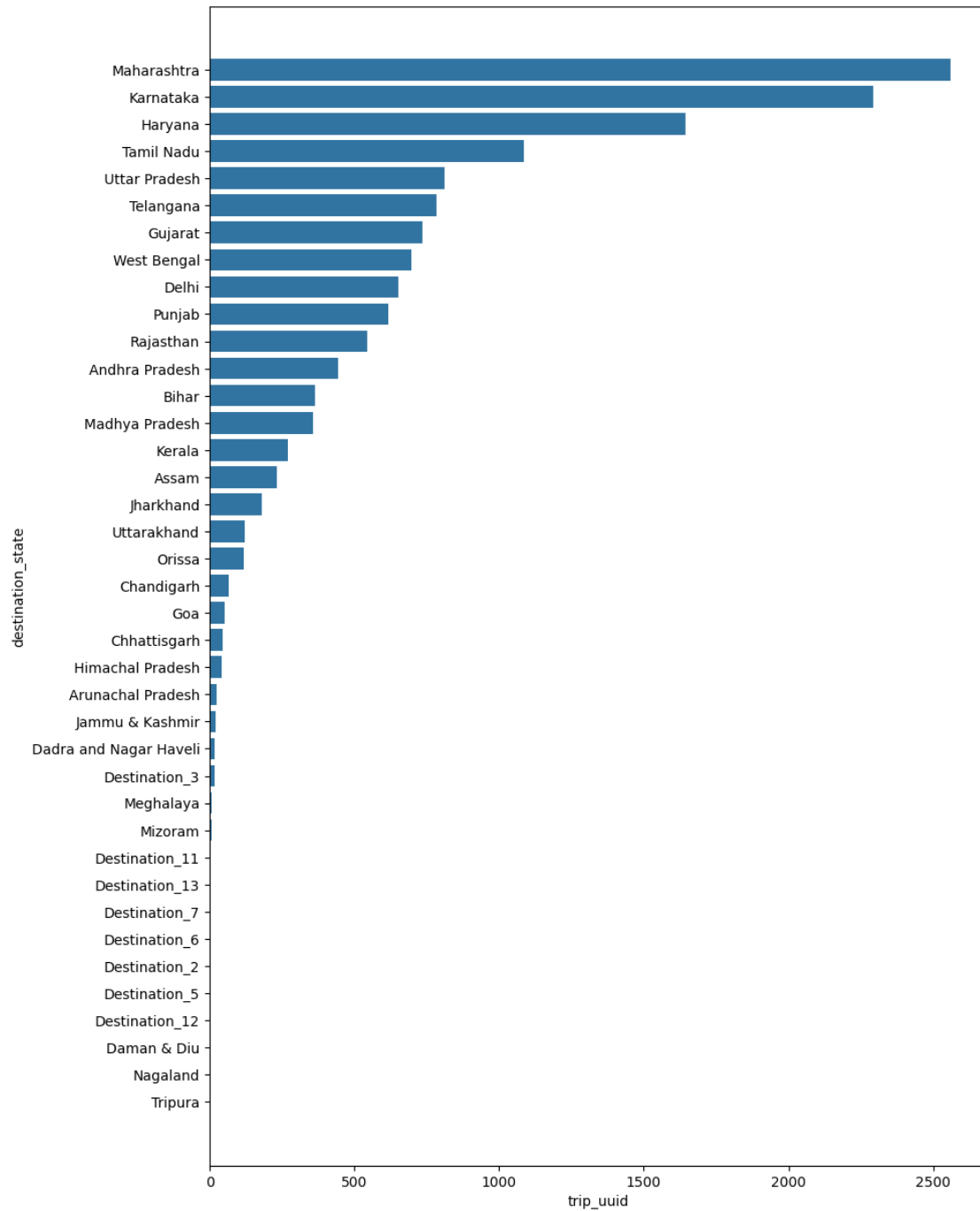
I am interested to know what is the distribution of number of trips which ended in different states

```
[83]: df_destination_state = df2.groupby(by = 'destination_state')['trip_uuid'].  
      ↪count().to_frame().reset_index()  
df_destination_state['perc'] = np.round(df_destination_state['trip_uuid'] * 100/  
      ↪ df_destination_state['trip_uuid'].sum(), 2)  
df_destination_state = df_destination_state.sort_values(by = 'trip_uuid',  
      ↪ascending = False)  
df_destination_state.head()
```

```
[83]: destination_state  trip_uuid  perc
      26      Maharashtra      2561  17.28
      23      Karnataka      2294  15.48
      19      Haryana      1643  11.09
      33      Tamil Nadu      1084   7.32
      36      Uttar Pradesh      811   5.47
```

```
[84]: plt.figure(figsize = (10, 15))
      sns.barplot(data = df_destination_state,
                  x = df_destination_state['trip_uuid'],
                  y = df_destination_state['destination_state'])
      plt.plot()
```

```
[84]: []
```



- It can be seen in the above plot that maximum trips ended in Maharashtra state followed by Karnataka, Haryana, Tamil Nadu and Uttar Pradesh. That means that the number of orders placed in these states is significantly high in these states.

I am interested to know top 30 cities based on the number of trips ended in different cities

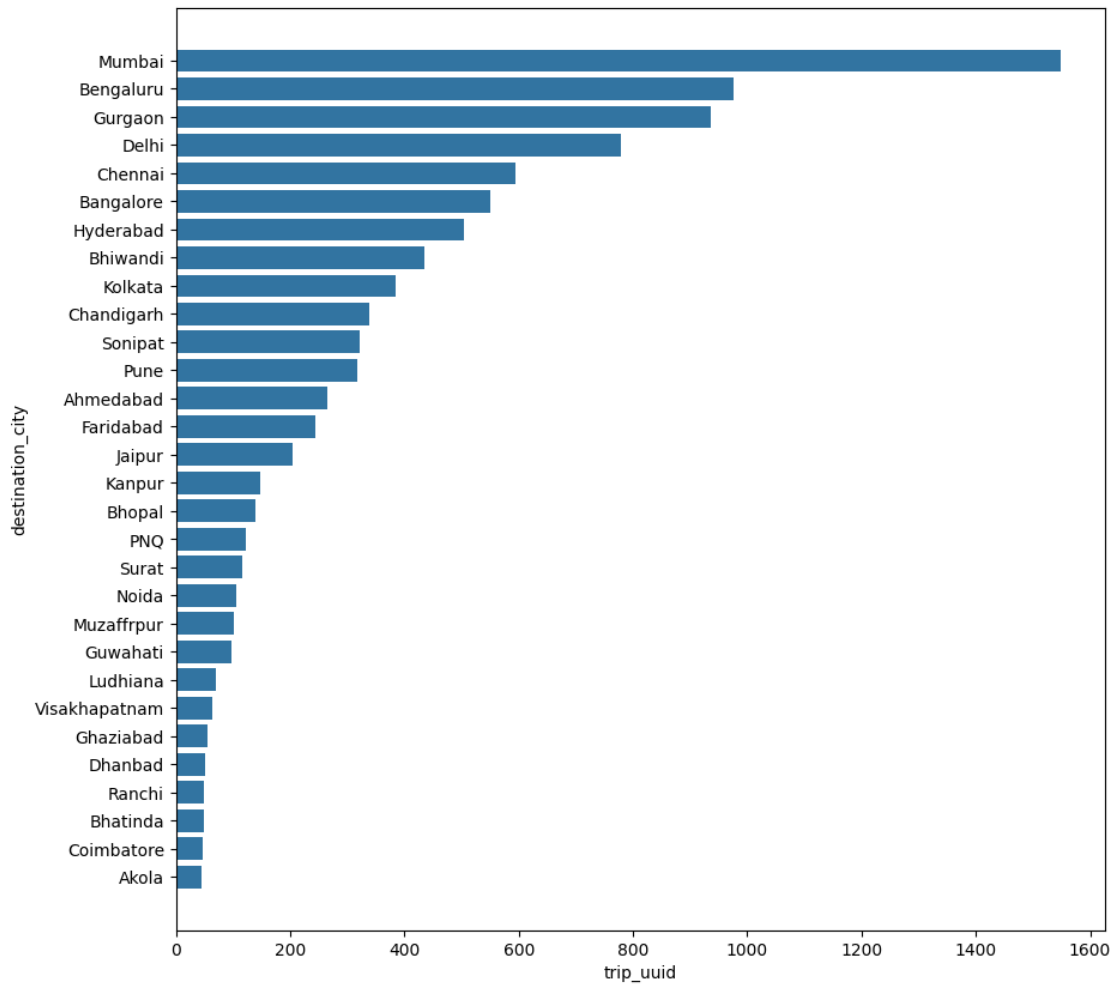
```
[85]: df_destination_city = df2.groupby(by = 'destination_city')['trip_uuid'].count().
      ↪to_frame().reset_index()
df_destination_city['perc'] = np.round(df_destination_city['trip_uuid'] * 100/
      ↪df_destination_city['trip_uuid'].sum(), 2)
df_destination_city = df_destination_city.sort_values(by = 'trip_uuid',
      ↪ascending = False)[:30]
df_destination_city
```

```
[85]:
```

	destination_city	trip_uuid	perc
516	Mumbai	1548	10.45
96	Bengaluru	975	6.58
283	Gurgaon	936	6.32
200	Delhi	778	5.25
163	Chennai	595	4.02
72	Bangalore	551	3.72
309	Hyderabad	503	3.39
115	Bhiwandi	434	2.93
419	Kolkata	384	2.59
158	Chandigarh	339	2.29
725	Sonipat	322	2.17
613	Pune	317	2.14
4	Ahmedabad	265	1.79
243	Faridabad	244	1.65
319	Jaipur	205	1.38
372	Kanpur	148	1.00
117	Bhopal	139	0.94
560	PNQ	122	0.82
740	Surat	117	0.79
553	Noida	106	0.72
522	Muzaffarpur	102	0.69
285	Guwahati	98	0.66
449	Ludhiana	70	0.47
798	Visakhapatnam	64	0.43
260	Ghaziabad	56	0.38
209	Dhanbad	50	0.34
640	Ranchi	49	0.33
110	Bhatinda	48	0.32
183	Coimbatore	47	0.32
9	Akola	45	0.30

```
[86]: plt.figure(figsize = (10, 10))
sns.barplot(data = df_destination_city,
            x = df_destination_city['trip_uuid'],
            y = df_destination_city['destination_city'])
plt.plot()
```

```
[86]: []
```



- It can be seen in the above plot that maximum trips ended in Mumbai city followed by Bengaluru, Gurgaon, Delhi and Chennai. That means that the number of orders placed in these cities is significantly high.

```
[88]: numerical_columns = ['od_total_time', 'start_scan_to_end_scan',
    ↪ 'actual_distance_to_destination',
    ↪ 'actual_time', 'osrm_time', 'osrm_distance',
    ↪ 'segment_actual_time',
    ↪ 'segment_osrm_time', 'segment_osrm_distance']
sns.pairplot(data = df2,
    vars = numerical_columns,
    kind = 'reg',
    hue = 'route_type',
    markers = '.')
plt.plot()
```

[88]: []



```
[89]: df_corr = df2[numerical_columns].corr()
df_corr
```

```
[89]:
```

	od_total_time	start_scan_to_end_scan	\
od_total_time	1.000000	0.999999	
start_scan_to_end_scan	0.999999	1.000000	
actual_distance_to_destination	0.918222	0.918308	
actual_time	0.961094	0.961147	
osrm_time	0.926516	0.926571	
osrm_distance	0.924219	0.924299	
segment_actual_time	0.961119	0.961171	
segment_osrm_time	0.918490	0.918561	

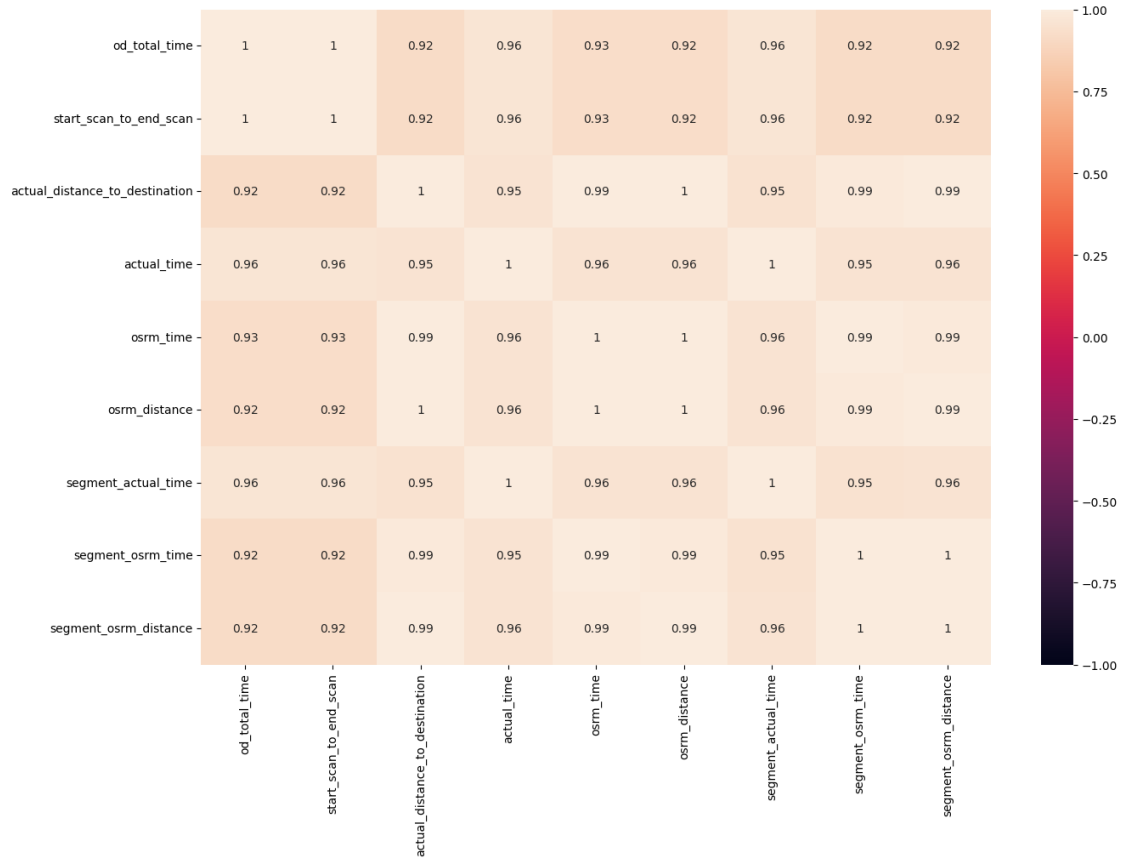
segment_osrm_distance	0.919199	0.919291	
-----------------------	----------	----------	--

	actual_distance_to_destination	actual_time	\
od_total_time	0.918222	0.961094	
start_scan_to_end_scan	0.918308	0.961147	
actual_distance_to_destination	1.000000	0.953757	
actual_time	0.953757	1.000000	
osrm_time	0.993561	0.958593	
osrm_distance	0.997264	0.959214	
segment_actual_time	0.952821	0.999989	
segment_osrm_time	0.987538	0.953872	
segment_osrm_distance	0.993061	0.956967	

	osrm_time	osrm_distance	segment_actual_time	\
od_total_time	0.926516	0.924219	0.961119	
start_scan_to_end_scan	0.926571	0.924299	0.961171	
actual_distance_to_destination	0.993561	0.997264	0.952821	
actual_time	0.958593	0.959214	0.999989	
osrm_time	1.000000	0.997580	0.957765	
osrm_distance	0.997580	1.000000	0.958353	
segment_actual_time	0.957765	0.958353	1.000000	
segment_osrm_time	0.993259	0.991798	0.953039	
segment_osrm_distance	0.991608	0.994710	0.956106	

	segment_osrm_time	segment_osrm_distance
od_total_time	0.918490	0.919199
start_scan_to_end_scan	0.918561	0.919291
actual_distance_to_destination	0.987538	0.993061
actual_time	0.953872	0.956967
osrm_time	0.993259	0.991608
osrm_distance	0.991798	0.994710
segment_actual_time	0.953039	0.956106
segment_osrm_time	1.000000	0.996092
segment_osrm_distance	0.996092	1.000000

```
[101]: plt.figure(figsize=(15, 10))
sns.heatmap(data=df_corr, vmin=-1, vmax=1, annot=True)
plt.show()
```



- Very High Correlation (> 0.9) exists between columns all the numerical columns specified above

0.1.12 3. In-depth analysis and feature engineering:

Compare the difference between od_total_time and start_scan_to_end_scan. Do hypothesis testing/ Visual analysis to check. STEP-1 : Set up Null Hypothesis

- Null Hypothesis (H_0) - od_total_time (Total Trip Time) and start_scan_to_end_scan (Expected total trip time) are same.
- Alternate Hypothesis (H_A) - od_total_time (Total Trip Time) and start_scan_to_end_scan (Expected total trip time) are different.

STEP-2 : Checking for basic assumptions for the hypothesis

- Distribution check using QQ Plot
- Homogeneity of Variances using Lavene's test

STEP-3: Define Test statistics; Distribution of T under H_0 .

- If the assumptions of T Test are met then we can proceed performing T Test for independent samples else we will perform the non parametric test equivalent to T Test for independent

sample i.e., Mann-Whitney U rank test for two independent samples.

STEP-4: Compute the p-value and fix value of alpha.

- We set our alpha to be 0.05

STEP-5: Compare p-value and alpha.

- Based on p-value, we will accept or reject H0.

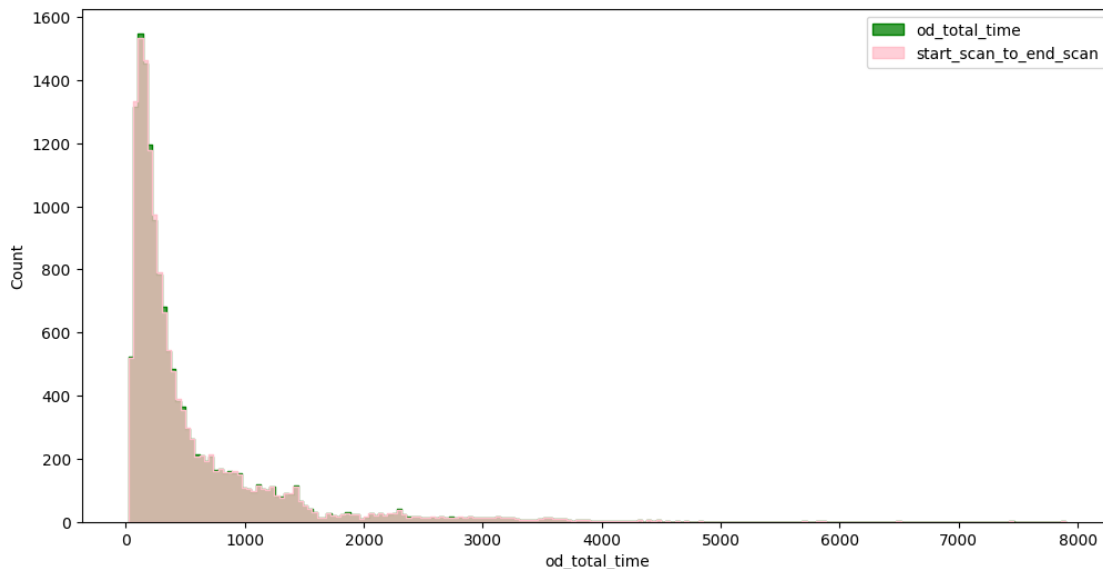
p-val > alpha : Accept H0

p-val < alpha : Reject H0

- Visual Tests to know if the samples follow normal distribution

```
[102]: plt.figure(figsize = (12, 6))
sns.histplot(df2['od_total_time'], element = 'step', color = 'green')
sns.histplot(df2['start_scan_to_end_scan'], element = 'step', color = 'pink')
plt.legend(['od_total_time', 'start_scan_to_end_scan'])
plt.plot()
```

[102]: []



- Distribution check using QQ Plot

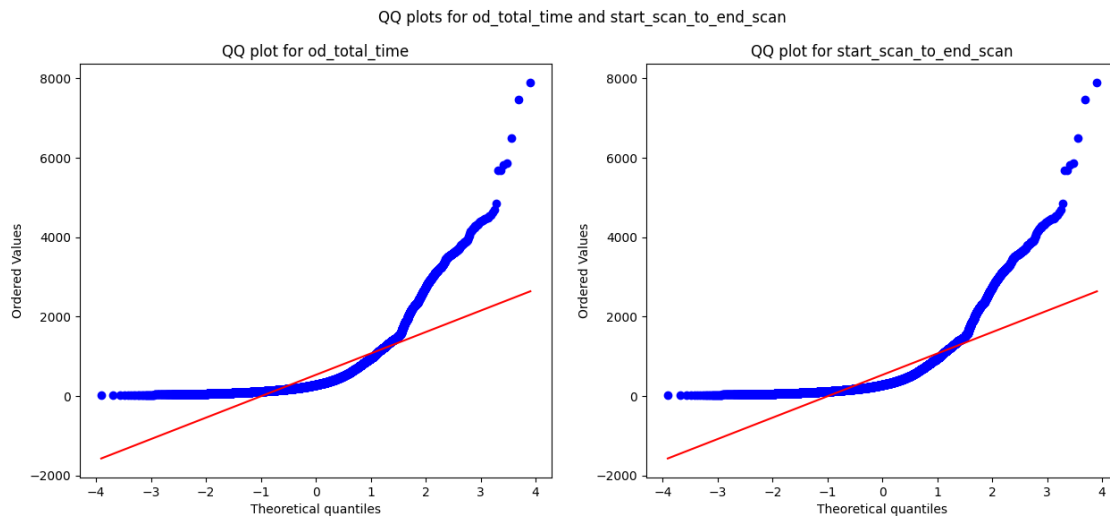
```
[103]: plt.figure(figsize = (15, 6))
plt.subplot(1, 2, 1)
plt.suptitle('QQ plots for od_total_time and start_scan_to_end_scan')
spy.probplot(df2['od_total_time'], plot = plt, dist = 'norm')
plt.title('QQ plot for od_total_time')
plt.subplot(1, 2, 2)
```

```

spy.probplot(df2['start_scan_to_end_scan'], plot = plt, dist = 'norm')
plt.title('QQ plot for start_scan_to_end_scan')
plt.plot()

```

[103]: []



It can be seen from the above plots that the samples do not come from normal distribution.

- Applying Shapiro-Wilk test for normality

Ho : The sample follows normal distribution H1 : The sample does not follow normal distribution

alpha = 0.05

Test Statistics : Shapiro-Wilk test for normality

```

[104]: test_stat, p_value = spy.shapiro(df2['od_total_time'].sample(5000))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')

```

p-value 9.633348436255193e-71

The sample does not follow normal distribution

```

[105]: test_stat, p_value = spy.shapiro(df2['start_scan_to_end_scan'].sample(5000))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:

```

```
print('The sample follows normal distribution')
```

p-value 1.5058388673783188e-71

The sample does not follow normal distribution

- Transforming the data using boxcox transformation to check if the transformed data follows normal distribution.

```
[106]: transformed_od_total_time = spy.boxcox(df2['od_total_time'])[0]
test_stat, p_value = spy.shapiro(transformed_od_total_time)
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 7.500452011421218e-25

The sample does not follow normal distribution

```
[107]: transformed_start_scan_to_end_scan = spy.
↳boxcox(df2['start_scan_to_end_scan'])[0]
test_stat, p_value = spy.shapiro(transformed_start_scan_to_end_scan)
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 1.0563381880973338e-24

The sample does not follow normal distribution

- Even after applying the boxcox transformation on each of the “od_total_time” and “start_scan_to_end_scan” columns, the distributions do not follow normal distribution.
- Homogeneity of Variances using Lavene’s test

```
[108]: # Null Hypothesis(H0) - Homogenous Variance

# Alternate Hypothesis(HA) - Non Homogenous Variance

test_stat, p_value = spy.levene(df2['od_total_time'],
↳df2['start_scan_to_end_scan'])
print('p-value', p_value)
if p_value < 0.05:
    print('The samples do not have Homogenous Variance')
else:
    print('The samples have Homogenous Variance ')
```

p-value 0.9668007217581142

The samples have Homogenous Variance

Since the samples are not normally distributed, T-Test cannot be applied here, we can perform its non parametric equivalent test i.e., Mann-Whitney U rank test for two independent samples.

```
[109]: test_stat, p_value = spy.mannwhitneyu(df2['od_total_time'],  
      ↪df2['start_scan_to_end_scan'])  
print('P-value :',p_value)
```

P-value : 0.7815123224221716

Since $p\text{-value} > \alpha$ therefore it can be concluded that `od_total_time` and `start_scan_to_end_scan` are similar.

Do hypothesis testing / visual analysis between `actual_time` aggregated value and `OSRM` time aggregated value (aggregated values are the values you'll get after merging the rows on the basis of `trip_uuid`)

```
[110]: df2[['actual_time', 'osrm_time']].describe()
```

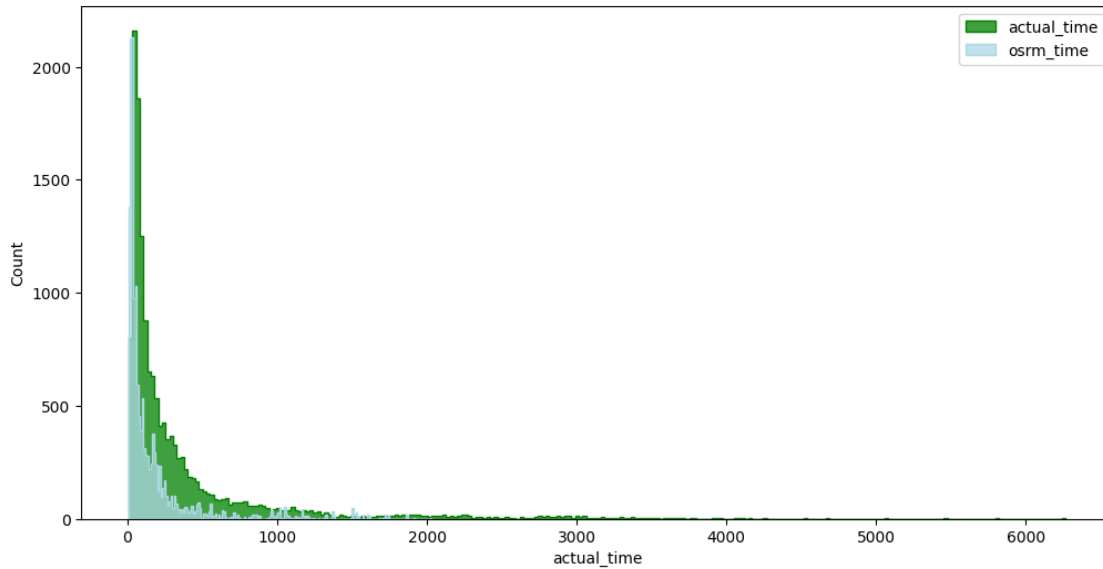
```
[110]:
```

	actual_time	osrm_time
count	14817.000000	14817.000000
mean	357.143768	161.384018
std	561.396179	271.360992
min	9.000000	6.000000
25%	67.000000	29.000000
50%	149.000000	60.000000
75%	370.000000	168.000000
max	6265.000000	2032.000000

- Visual Tests to know if the samples follow normal distribution

```
[111]: plt.figure(figsize = (12, 6))  
sns.histplot(df2['actual_time'], element = 'step', color = 'green')  
sns.histplot(df2['osrm_time'], element = 'step', color = 'lightblue')  
plt.legend(['actual_time', 'osrm_time'])  
plt.plot()
```

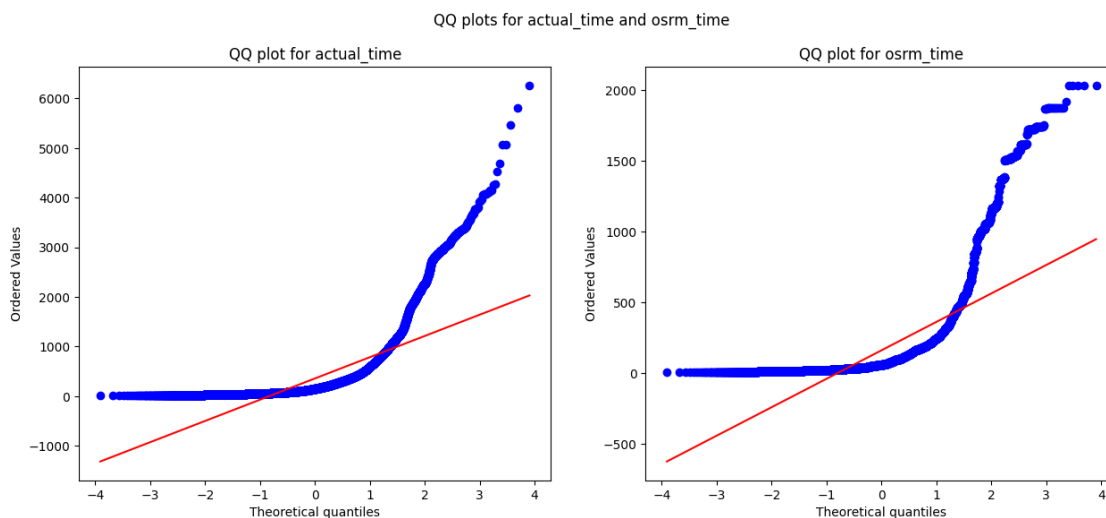
```
[111]: []
```



- Distribution check using QQ Plot

```
[112]: plt.figure(figsize = (15, 6))
plt.subplot(1, 2, 1)
plt.suptitle('QQ plots for actual_time and osrm_time')
spy.probplot(df2['actual_time'], plot = plt, dist = 'norm')
plt.title('QQ plot for actual_time')
plt.subplot(1, 2, 2)
spy.probplot(df2['osrm_time'], plot = plt, dist = 'norm')
plt.title('QQ plot for osrm_time')
plt.plot()
```

[112]: []



It can be seen from the above plots that the samples do not come from normal distribution.

- Applying Shapiro-Wilk test for normality

Ho : The sample follows normal distribution H1 : The sample does not follow normal distribution

alpha = 0.05

Test Statistics : Shapiro-Wilk test for normality

```
[113]: test_stat, p_value = spy.shapiro(df2['actual_time'].sample(5000))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 2.3037320940781908e-76

The sample does not follow normal distribution

```
[114]: test_stat, p_value = spy.shapiro(df2['osrm_time'].sample(5000))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 1.7336979912260015e-77

The sample does not follow normal distribution

- Transforming the data using boxcox transformation to check if the transformed data follows normal distribution.

```
[115]: transformed_actual_time = spy.boxcox(df2['actual_time'])[0]
test_stat, p_value = spy.shapiro(transformed_actual_time)
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 1.060536186579569e-28

The sample does not follow normal distribution

```
[116]: transformed_osrm_time = spy.boxcox(df2['osrm_time'])[0]
test_stat, p_value = spy.shapiro(transformed_osrm_time)
print('p-value', p_value)
```

```

if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')

```

p-value 3.5256446740813525e-35

The sample does not follow normal distribution

- Even after applying the boxcox transformation on each of the “actual_time” and “osrm_time” columns, the distributions do not follow normal distribution.
- Homogeneity of Variances using Lavene’s test

```

[117]: # Null Hypothesis(H0) - Homogenous Variance

# Alternate Hypothesis(HA) - Non Homogenous Variance

test_stat, p_value = spy.levene(df2['actual_time'], df2['osrm_time'])
print('p-value', p_value)
if p_value < 0.05:
    print('The samples do not have Homogenous Variance')
else:
    print('The samples have Homogenous Variance ')

```

p-value 1.871297993683208e-220

The samples do not have Homogenous Variance

Since the samples do not follow any of the assumptions T-Test cannot be applied here, we can perform its non parametric equivalent test i.e., Mann-Whitney U rank test for two independent samples.

```

[118]: test_stat, p_value = spy.mannwhitneyu(df2['actual_time'], df2['osrm_time'])
print('p-value', p_value)
if p_value < 0.05:
    print('The samples are not similar')
else:
    print('The samples are similar ')

```

p-value 0.0

The samples are not similar

Since p-value < alpha therefore it can be concluded that actual_time and osrm_time are not similar.

Do hypothesis testing/ visual analysis between actual_time aggregated value and segment actual time aggregated value (aggregated values are the values you’ll get after merging the rows on the basis of trip_uuid)

```

[119]: df2[['actual_time', 'segment_actual_time']].describe()

```

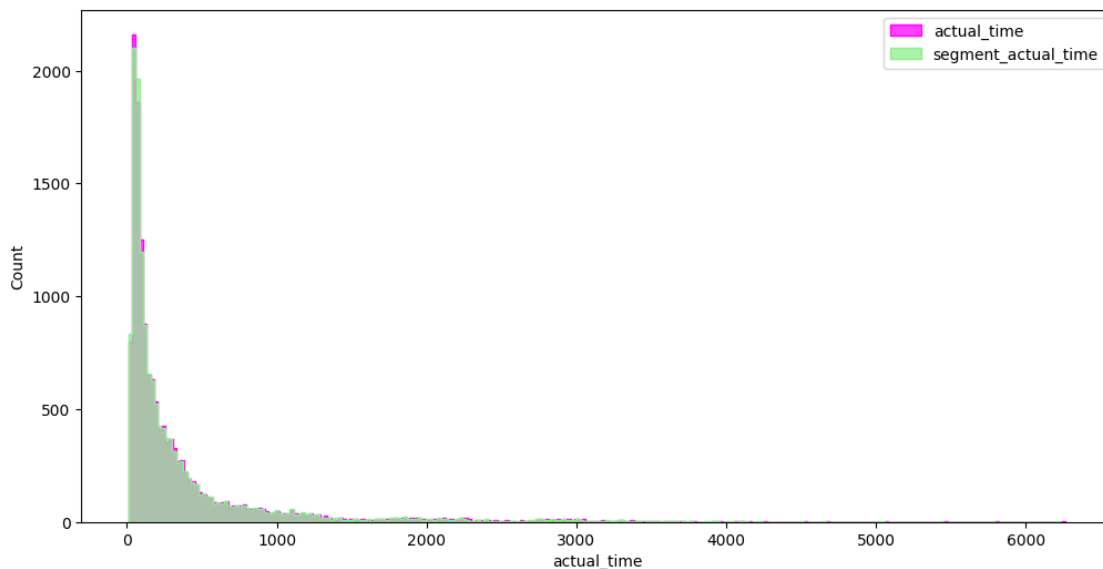
```
[119]:
```

	actual_time	segment_actual_time
count	14817.000000	14817.000000
mean	357.143768	353.892273
std	561.396179	556.247986
min	9.000000	9.000000
25%	67.000000	66.000000
50%	149.000000	147.000000
75%	370.000000	367.000000
max	6265.000000	6230.000000

- Visual Tests to know if the samples follow normal distribution

```
[120]: plt.figure(figsize = (12, 6))
sns.histplot(df2['actual_time'], element = 'step', color = 'magenta')
sns.histplot(df2['segment_actual_time'], element = 'step', color = 'lightgreen')
plt.legend(['actual_time', 'segment_actual_time'])
plt.plot()
```

```
[120]: []
```

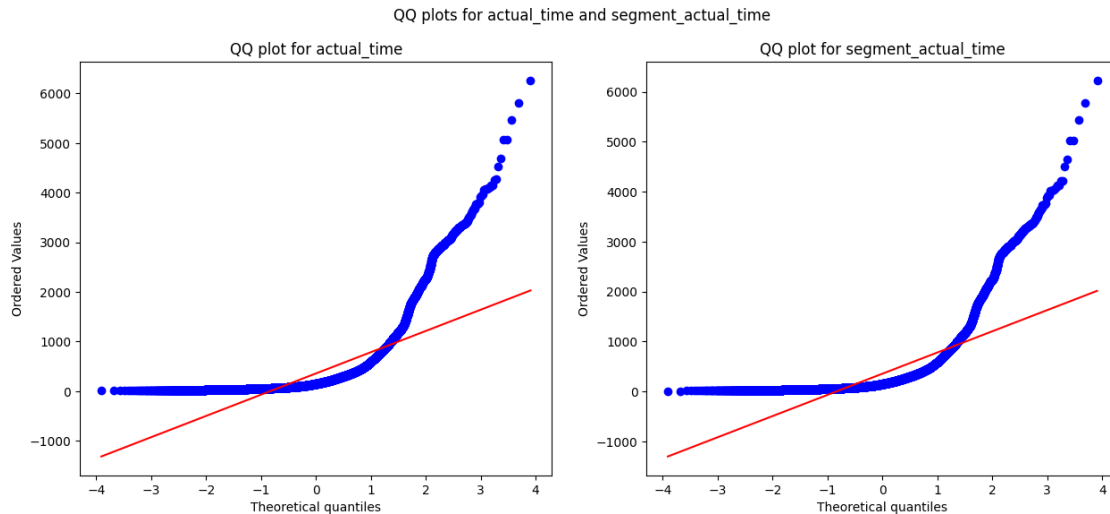


- Distribution check using QQ Plot

```
[121]: plt.figure(figsize = (15, 6))
plt.subplot(1, 2, 1)
plt.suptitle('QQ plots for actual_time and segment_actual_time')
spy.probplot(df2['actual_time'], plot = plt, dist = 'norm')
plt.title('QQ plot for actual_time')
plt.subplot(1, 2, 2)
spy.probplot(df2['segment_actual_time'], plot = plt, dist = 'norm')
```

```
plt.title('QQ plot for segment_actual_time')
plt.plot()
```

[121]: []



It can be seen from the above plots that the samples do not come from normal distribution.

- Applying Shapiro-Wilk test for normality

Ho : The sample follows normal distribution H1 : The sample does not follow normal distribution

alpha = 0.05

Test Statistics : Shapiro-Wilk test for normality

```
[123]: test_stat, p_value = spy.shapiro(df2['actual_time'].sample(5000))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 2.2212953648338487e-76

The sample does not follow normal distribution

```
[124]: test_stat, p_value = spy.shapiro(df2['segment_actual_time'].sample(5000))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 2.94033116576506e-76

The sample does not follow normal distribution

- Transforming the data using boxcox transformation to check if the transformed data follows normal distribution.

```
[125]: transformed_actual_time = spy.boxcox(df2['actual_time'])[0]
test_stat, p_value = spy.shapiro(transformed_actual_time)
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 1.060536186579569e-28

The sample does not follow normal distribution

```
[126]: transformed_segment_actual_time = spy.boxcox(df2['segment_actual_time'])[0]
test_stat, p_value = spy.shapiro(transformed_segment_actual_time)
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 5.938231499945463e-29

The sample does not follow normal distribution

- Even after applying the boxcox transformation on each of the “actual_time” and “segment_actual_time” columns, the distributions do not follow normal distribution.
- Homogeneity of Variances using Lavene’s test

```
[127]: # Null Hypothesis(H0) - Homogenous Variance

# Alternate Hypothesis(HA) - Non Homogenous Variance

test_stat, p_value = spy.levene(df2['actual_time'], df2['segment_actual_time'])
print('p-value', p_value)

if p_value < 0.05:
    print('The samples do not have Homogenous Variance')
else:
    print('The samples have Homogenous Variance ')
```

p-value 0.6955022668700895

The samples have Homogenous Variance

Since the samples do not come from normal distribution T-Test cannot be applied here, we can perform its non parametric equivalent test i.e., Mann-Whitney U rank test for two independent samples.

```
[128]: test_stat, p_value = spy.mannwhitneyu(df2['actual_time'],
      ↪df2['segment_actual_time'])
print('p-value', p_value)
if p_value < 0.05:
    print('The samples are not similar')
else:
    print('The samples are similar ')
```

p-value 0.4164235159622476
The samples are similar

Since $p\text{-value} > \alpha$ therefore it can be concluded that `actual_time` and `segment_actual_time` are similar.

Do hypothesis testing/ visual analysis between osrm distance aggregated value and segment osrm distance aggregated value (aggregated values are the values you'll get after merging the rows on the basis of `trip_uid`)

```
[129]: df2[['osrm_distance', 'segment_osrm_distance']].describe()
```

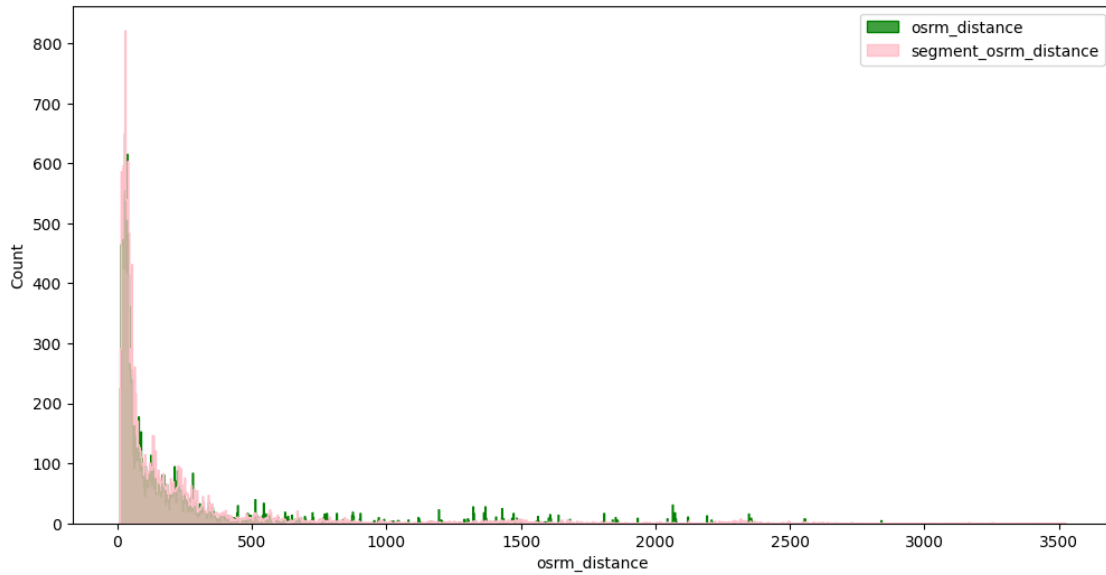
```
[129]:
```

	osrm_distance	segment_osrm_distance
count	14817.000000	14817.000000
mean	204.344711	223.201157
std	370.395569	416.628387
min	9.072900	9.072900
25%	30.819201	32.654499
50%	65.618805	70.154404
75%	208.475006	218.802399
max	2840.081055	3523.632324

- Visual Tests to know if the samples follow normal distribution

```
[130]: plt.figure(figsize = (12, 6))
sns.histplot(df2['osrm_distance'], element = 'step', color = 'green', bins =
      ↪1000)
sns.histplot(df2['segment_osrm_distance'], element = 'step', color = 'pink',
      ↪bins = 1000)
plt.legend(['osrm_distance', 'segment_osrm_distance'])
plt.plot()
```

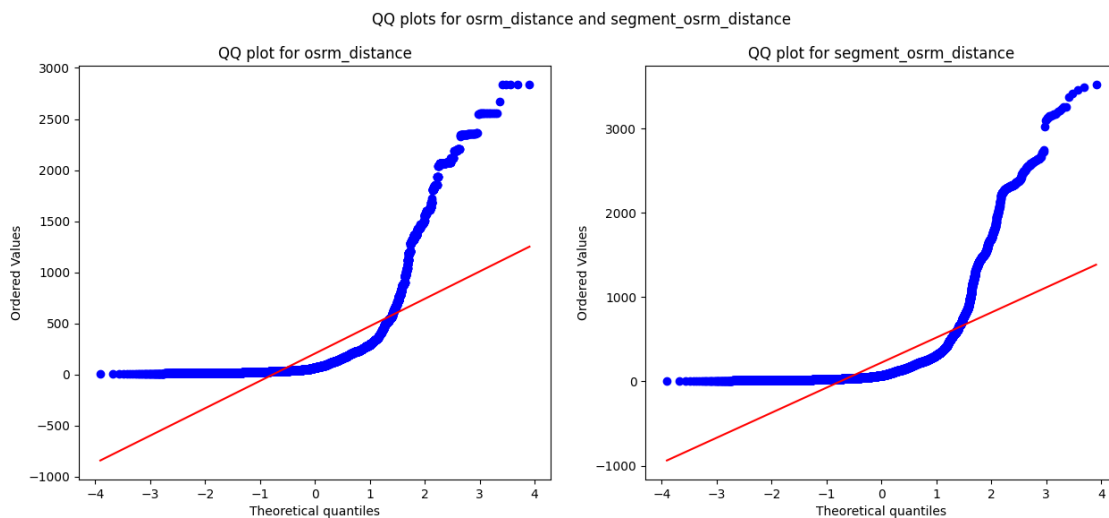
```
[130]: []
```



- Distribution check using QQ Plot

```
[131]: plt.figure(figsize = (15, 6))
plt.subplot(1, 2, 1)
plt.suptitle('QQ plots for osrm_distance and segment_osrm_distance')
spy.probplot(df2['osrm_distance'], plot = plt, dist = 'norm')
plt.title('QQ plot for osrm_distance')
plt.subplot(1, 2, 2)
spy.probplot(df2['segment_osrm_distance'], plot = plt, dist = 'norm')
plt.title('QQ plot for segment_osrm_distance')
plt.plot()
```

[131]: []



It can be seen from the above plots that the samples do not come from normal distribution.

- Applying Shapiro-Wilk test for normality

Ho : The sample follows normal distribution H1 : The sample does not follow normal distribution

alpha = 0.05

Test Statistics : Shapiro-Wilk test for normality

```
[132]: test_stat, p_value = spy.shapiro(df2['osrm_distance'].sample(5000))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 3.1051234151607315e-79

The sample does not follow normal distribution

```
[133]: test_stat, p_value = spy.shapiro(df2['segment_osrm_distance'].sample(5000))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 1.0291469287256529e-79

The sample does not follow normal distribution

- Transforming the data using boxcox transformation to check if the transformed data follows normal distribution.

```
[134]: transformed_osrm_distance = spy.boxcox(df2['osrm_distance'])[0]
test_stat, p_value = spy.shapiro(transformed_osrm_distance)
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 7.106940835518284e-41

The sample does not follow normal distribution

```
[135]: transformed_segment_osrm_distance = spy.boxcox(df2['segment_osrm_distance'])[0]
test_stat, p_value = spy.shapiro(transformed_segment_osrm_distance)
print('p-value', p_value)
```

```

if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')

```

p-value 3.0563904695481e-38

The sample does not follow normal distribution

- Even after applying the boxcox transformation on each of the “osrm_distance” and “segment_osrm_distance” columns, the distributions do not follow normal distribution.
- Homogeneity of Variances using Lavene’s test

```

[136]: # Null Hypothesis(H0) - Homogenous Variance

# Alternate Hypothesis(HA) - Non Homogenous Variance

test_stat, p_value = spy.levene(df2['osrm_distance'],
    ↪df2['segment_osrm_distance'])
print('p-value', p_value)

if p_value < 0.05:
    print('The samples do not have Homogenous Variance')
else:
    print('The samples have Homogenous Variance ')

```

p-value 0.00020976355663265204

The samples do not have Homogenous Variance

Since the samples do not follow any of the assumptions, T-Test cannot be applied here. We can perform its non parametric equivalent test i.e., Mann-Whitney U rank test for two independent samples.

```

[137]: test_stat, p_value = spy.mannwhitneyu(df2['osrm_distance'],
    ↪df2['segment_osrm_distance'])
print('p-value', p_value)
if p_value < 0.05:
    print('The samples are not similar')
else:
    print('The samples are similar ')

```

p-value 9.509410818847664e-07

The samples are not similar

Since $p\text{-value} < \alpha$ therefore it can be concluded that osrm_distance and segment_osrm_distance are not similar.

Do hypothesis testing/ visual analysis between osrm time aggregated value and segment osrm time aggregated value (aggregated values are the values you’ll get after merging the rows on the basis of trip_uuid)

```
[138]: df2[['osrm_time', 'segment_osrm_time']].describe().T
```

```
[138]:
```

	count	mean	std	min	25%	50%	75%	\
osrm_time	14817.0	161.384018	271.360992	6.0	29.0	60.0	168.0	
segment_osrm_time	14817.0	180.949783	314.542053	6.0	31.0	65.0	185.0	


```

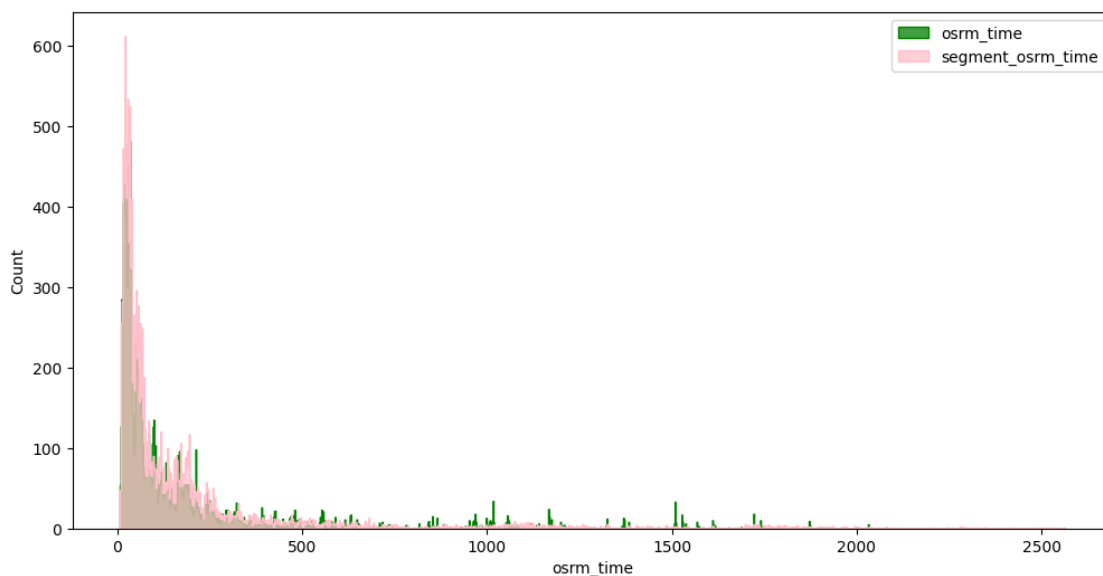
                                max
osrm_time                      2032.0
segment_osrm_time              2564.0

```

- Visual Tests to know if the samples follow normal distribution

```
[139]: plt.figure(figsize = (12, 6))
sns.histplot(df2['osrm_time'], element = 'step', color = 'green', bins = 1000)
sns.histplot(df2['segment_osrm_time'], element = 'step', color = 'pink', bins = 1000)
plt.legend(['osrm_time', 'segment_osrm_time'])
plt.plot()
```

```
[139]: []
```



- Distribution check using QQ Plot

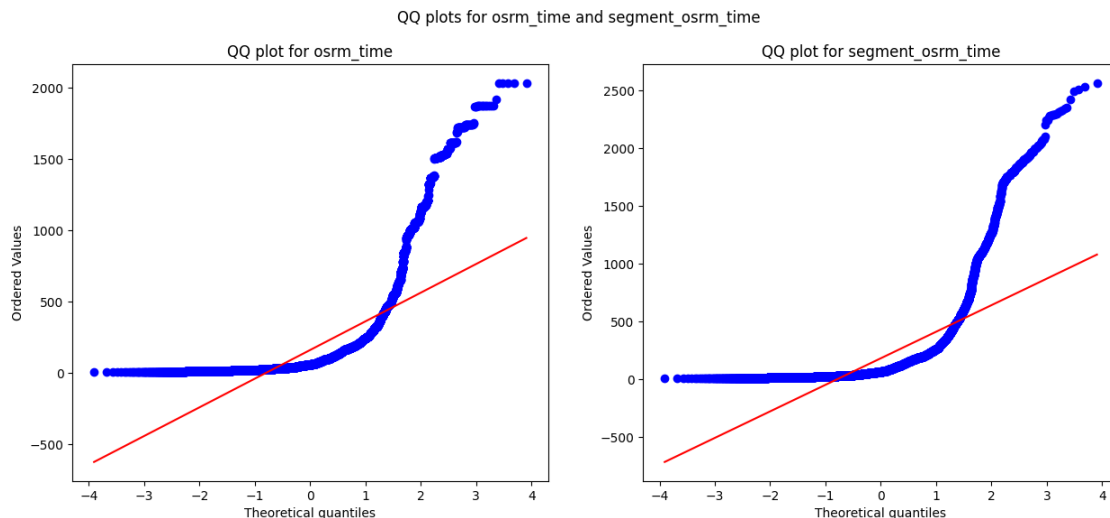
```
[140]: plt.figure(figsize = (15, 6))
plt.subplot(1, 2, 1)
plt.suptitle('QQ plots for osrm_time and segment_osrm_time')
spy.probplot(df2['osrm_time'], plot = plt, dist = 'norm')
plt.title('QQ plot for osrm_time')
plt.subplot(1, 2, 2)
```

```

spy.probplot(df2['segment_osrm_time'], plot = plt, dist = 'norm')
plt.title('QQ plot for segment_osrm_time')
plt.plot()

```

[140]: []



It can be seen from the above plots that the samples do not come from normal distribution. Applying Shapiro-Wilk test for normality

Ho : The sample follows normal distribution H1 : The sample does not follow normal distribution

alpha = 0.05

Test Statistics : Shapiro-Wilk test for normality

```

[141]: test_stat, p_value = spy.shapiro(df2['osrm_time'].sample(5000))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')

```

p-value 4.0112777520877234e-78

The sample does not follow normal distribution

```

[142]: test_stat, p_value = spy.shapiro(df2['segment_osrm_time'].sample(5000))
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')

```

p-value 5.856342558511798e-79

The sample does not follow normal distribution

- Transforming the data using boxcox transformation to check if the transformed data follows normal distribution

```
[143]: transformed_osrm_time = spy.boxcox(df2['osrm_time'])[0]
test_stat, p_value = spy.shapiro(transformed_osrm_time)
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 3.5256446740813525e-35

The sample does not follow normal distribution

```
[144]: transformed_segment_osrm_time = spy.boxcox(df2['segment_osrm_time'])[0]
test_stat, p_value = spy.shapiro(transformed_segment_osrm_time)
print('p-value', p_value)
if p_value < 0.05:
    print('The sample does not follow normal distribution')
else:
    print('The sample follows normal distribution')
```

p-value 4.899366703430711e-34

The sample does not follow normal distribution

- Even after applying the boxcox transformation on each of the “osrm_time” and “segment_osrm_time” columns, the distributions do not follow normal distribution.
- Homogeneity of Variances using Lavene’s test

```
[145]: # Null Hypothesis(H0) - Homogenous Variance

# Alternate Hypothesis(HA) - Non Homogenous Variance

test_stat, p_value = spy.levene(df2['osrm_time'], df2['segment_osrm_time'])
print('p-value', p_value)

if p_value < 0.05:
    print('The samples do not have Homogenous Variance')
else:
    print('The samples have Homogenous Variance ')
```

p-value 8.349482669010088e-08

The samples do not have Homogenous Variance

Since the samples do not follow any of the assumptions, T-Test cannot be applied here. We can perform its non parametric equivalent test i.e., Mann-Whitney U rank test for two independent samples.

```
[146]: test_stat, p_value = spy.mannwhitneyu(df2['osrm_time'],
      ↪df2['segment_osrm_time'])
print('p-value', p_value)
if p_value < 0.05:
    print('The samples are not similar')
else:
    print('The samples are similar ')
```

p-value 2.2995370859748865e-08
The samples are not similar

Since $p\text{-value} < \alpha$ therefore it can be concluded that `osrm_time` and `segment_osrm_time` are not similar.

Find outliers in the numerical variables (you might find outliers in almost all the variables), and check it using visual analysis

```
[147]: numerical_columns = ['od_total_time', 'start_scan_to_end_scan',
      ↪'actual_distance_to_destination',
      ↪'actual_time', 'osrm_time', 'osrm_distance',
      ↪'segment_actual_time',
      ↪'segment_osrm_time', 'segment_osrm_distance']
df2[numerical_columns].describe().T
```

```
[147]:
```

	count	mean	std	min \
od_total_time	14817.0	531.697630	658.868223	23.460000
start_scan_to_end_scan	14817.0	530.810016	658.705957	23.000000
actual_distance_to_destination	14817.0	164.477829	305.388153	9.002461
actual_time	14817.0	357.143768	561.396179	9.000000
osrm_time	14817.0	161.384018	271.360992	6.000000
osrm_distance	14817.0	204.344711	370.395569	9.072900
segment_actual_time	14817.0	353.892273	556.247986	9.000000
segment_osrm_time	14817.0	180.949783	314.542053	6.000000
segment_osrm_distance	14817.0	223.201157	416.628387	9.072900

	25%	50%	75% \
od_total_time	149.930000	280.770000	638.200000
start_scan_to_end_scan	149.000000	280.000000	637.000000
actual_distance_to_destination	22.837238	48.474072	164.583206
actual_time	67.000000	149.000000	370.000000
osrm_time	29.000000	60.000000	168.000000
osrm_distance	30.819201	65.618805	208.475006
segment_actual_time	66.000000	147.000000	367.000000
segment_osrm_time	31.000000	65.000000	185.000000
segment_osrm_distance	32.654499	70.154404	218.802399

	max
od_total_time	7898.550000

```

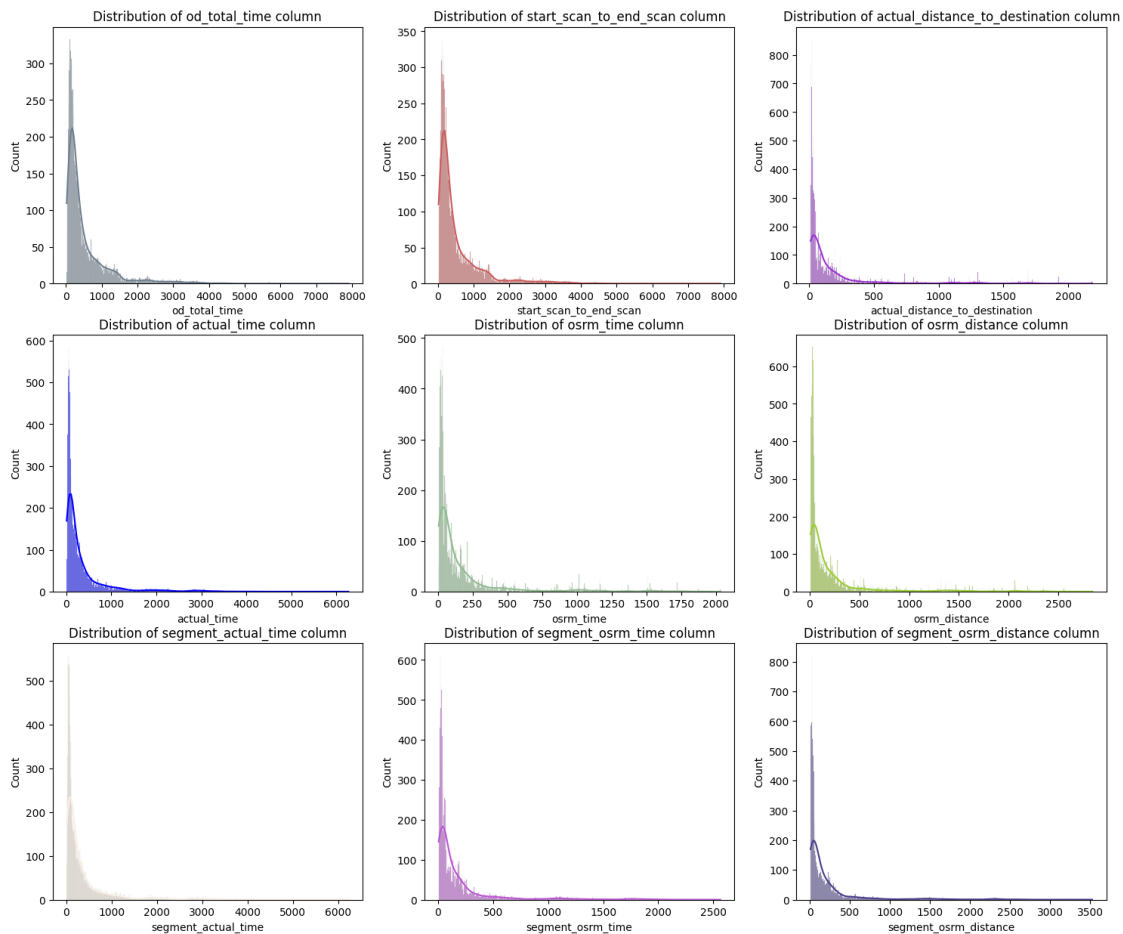
start_scan_to_end_scan      7898.000000
actual_distance_to_destination 2186.531738
actual_time                  6265.000000
osrm_time                    2032.000000
osrm_distance                2840.081055
segment_actual_time          6230.000000
segment_osrm_time            2564.000000
segment_osrm_distance        3523.632324

```

```

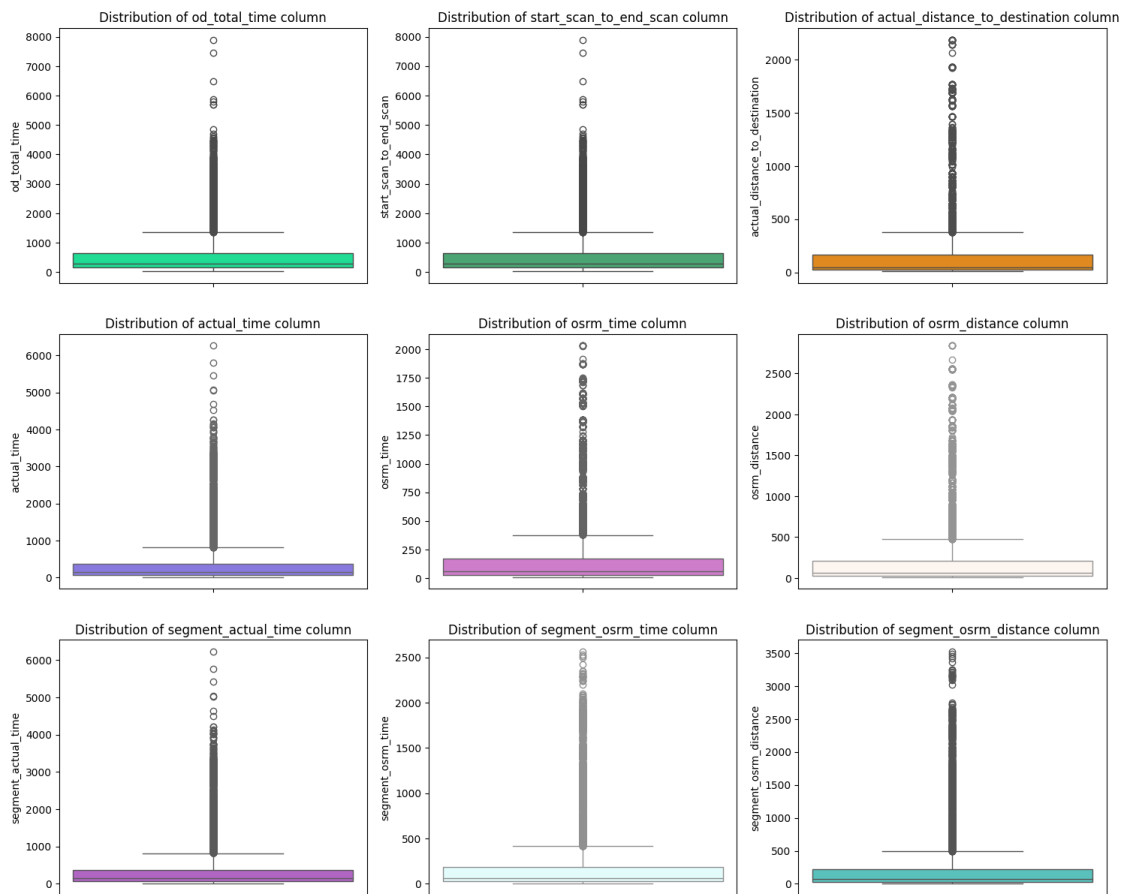
[148]: plt.figure(figsize = (18, 15))
for i in range(len(numerical_columns)):
    plt.subplot(3, 3, i + 1)
    clr = np.random.choice(list(mpl.colors.cnames))
    sns.histplot(df2[numerical_columns[i]], bins = 1000, kde = True, color = clr)
    plt.title(f"Distribution of {numerical_columns[i]} column")
    plt.plot()

```



- It can be inferred from the above plots that data in all the numerical columns are right skewed.

```
[149]: plt.figure(figsize = (18, 15))
for i in range(len(numerical_columns)):
    plt.subplot(3, 3, i + 1)
    clr = np.random.choice(list(mpl.colors.cnames))
    sns.boxplot(df2[numerical_columns[i]], color = clr)
    plt.title(f"Distribution of {numerical_columns[i]} column")
    plt.plot()
```



- It can be clearly seen in the above plots that there are outliers in all the numerical columns that need to be treated.

```
[150]: # Detecting Outliers

for i in numerical_columns:
    Q1 = np.quantile(df2[i], 0.25)
    Q3 = np.quantile(df2[i], 0.75)
    IQR = Q3 - Q1
```

```

LB = Q1 - 1.5 * IQR
UB = Q3 + 1.5 * IQR
outliers = df2.loc[(df2[i] < LB) | (df2[i] > UB)]
print('Column :', i)
print(f'Q1 : {Q1}')
print(f'Q3 : {Q3}')
print(f'IQR : {IQR}')
print(f'LB : {LB}')
print(f'UB : {UB}')
print(f'Number of outliers : {outliers.shape[0]}')
print('-----')

```

```

Column : od_total_time
Q1 : 149.93
Q3 : 638.2
IQR : 488.27000000000004
LB : -582.4750000000001
UB : 1370.605
Number of outliers : 1266
-----
Column : start_scan_to_end_scan
Q1 : 149.0
Q3 : 637.0
IQR : 488.0
LB : -583.0
UB : 1369.0
Number of outliers : 1267
-----
Column : actual_distance_to_destination
Q1 : 22.837238311767578
Q3 : 164.5832061767578
IQR : 141.7459716796875
LB : -189.78172302246094
UB : 377.2021484375
Number of outliers : 1449
-----
Column : actual_time
Q1 : 67.0
Q3 : 370.0
IQR : 303.0
LB : -387.5
UB : 824.5
Number of outliers : 1643
-----
Column : osrm_time
Q1 : 29.0
Q3 : 168.0
IQR : 139.0

```

```

LB : -179.5
UB : 376.5
Number of outliers : 1517
-----
Column : osrm_distance
Q1 : 30.81920051574707
Q3 : 208.47500610351562
IQR : 177.6558074951172
LB : -235.6645050048828
UB : 474.9587097167969
Number of outliers : 1524
-----
Column : segment_actual_time
Q1 : 66.0
Q3 : 367.0
IQR : 301.0
LB : -385.5
UB : 818.5
Number of outliers : 1643
-----
Column : segment_osrm_time
Q1 : 31.0
Q3 : 185.0
IQR : 154.0
LB : -200.0
UB : 416.0
Number of outliers : 1492
-----
Column : segment_osrm_distance
Q1 : 32.65449905395508
Q3 : 218.80239868164062
IQR : 186.1479034423828
LB : -246.56736755371094
UB : 498.0242614746094
Number of outliers : 1548
-----

```

The outliers present in our sample data can be the true outliers. It's best to remove outliers only when there is a sound reason for doing so. Some outliers represent natural variations in the population, and they should be left as is in the dataset.

0.1.13 Do one-hot encoding of categorical variables (like route_type)

```

[151]: # Get value counts before one-hot encoding

df2['route_type'].value_counts()

```

```
[151]: route_type
      Carting      8908
      FTL          5909
      Name: count, dtype: int64
```

```
[ ]: !pip install scikit-learn
```

```
[155]: # Perform one-hot encoding on categorical column route type
      from sklearn.preprocessing import LabelEncoder
      label_encoder = LabelEncoder()
      df2['route_type'] = label_encoder.fit_transform(df2['route_type'])
```

```
[156]: # Get value counts after one-hot encoding

      df2['route_type'].value_counts()
```

```
[156]: route_type
      0      8908
      1      5909
      Name: count, dtype: int64
```

```
[157]: # Get value counts of categorical variable 'data' before one-hot encoding

      df2['data'].value_counts()
```

```
[157]: data
      training    10654
      test        4163
      Name: count, dtype: int64
```

```
[158]: # Perform one-hot encoding on categorical variable 'data'
      label_encoder = LabelEncoder()
      df2['data'] = label_encoder.fit_transform(df2['data'])
```

```
[159]: # Get value counts after one-hot encoding

      df2['data'].value_counts()
```

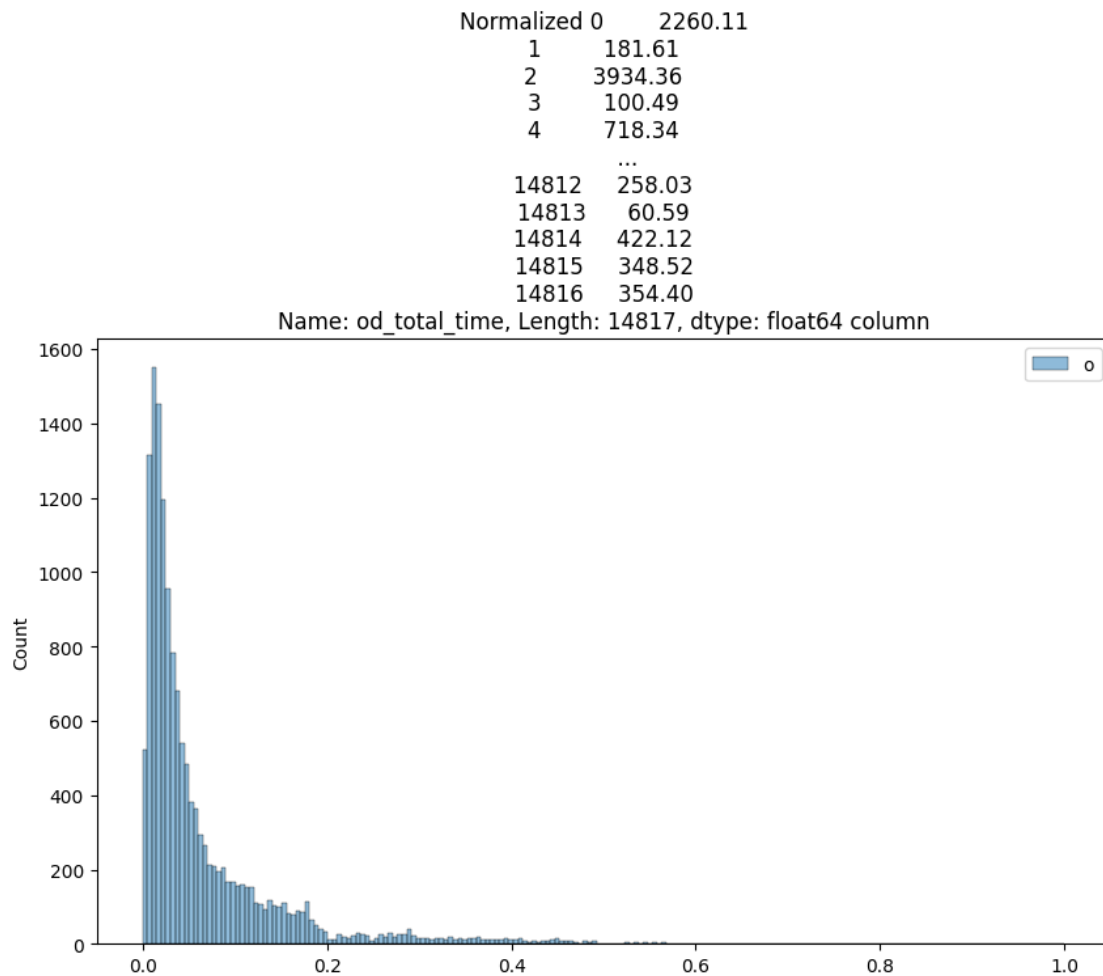
```
[159]: data
      1      10654
      0       4163
      Name: count, dtype: int64
```

0.1.14 Normalize/ Standardize the numerical features using MinMaxScaler or StandardScaler.

```
[160]: from sklearn.preprocessing import MinMaxScaler
```

```
[161]: plt.figure(figsize = (10, 6))
scaler = MinMaxScaler()
scaled = scaler.fit_transform(df2['od_total_time'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Normalized {df2['od_total_time']} column")
plt.legend('od_total_time')
plt.plot()
```

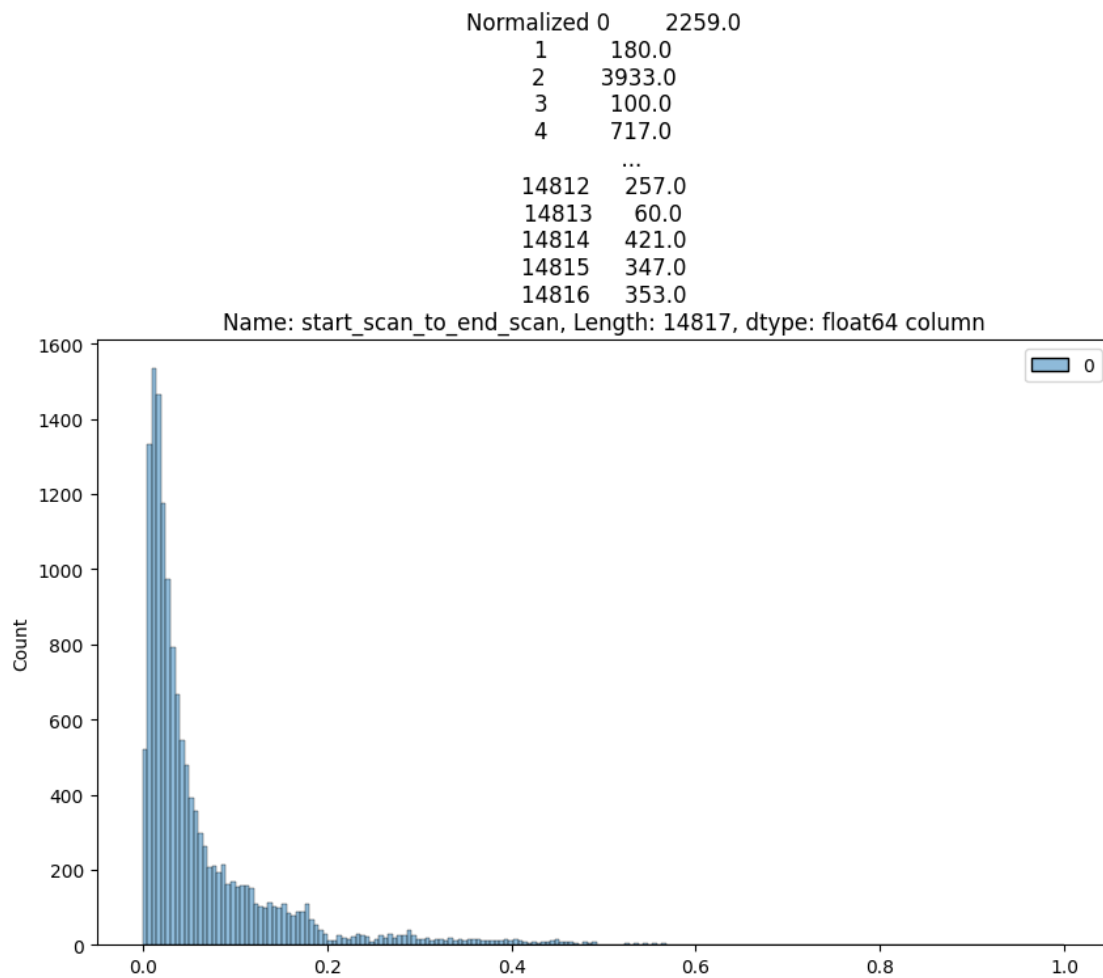
```
[161]: []
```



```
[162]: plt.figure(figsize = (10, 6))
scaler = MinMaxScaler()
```

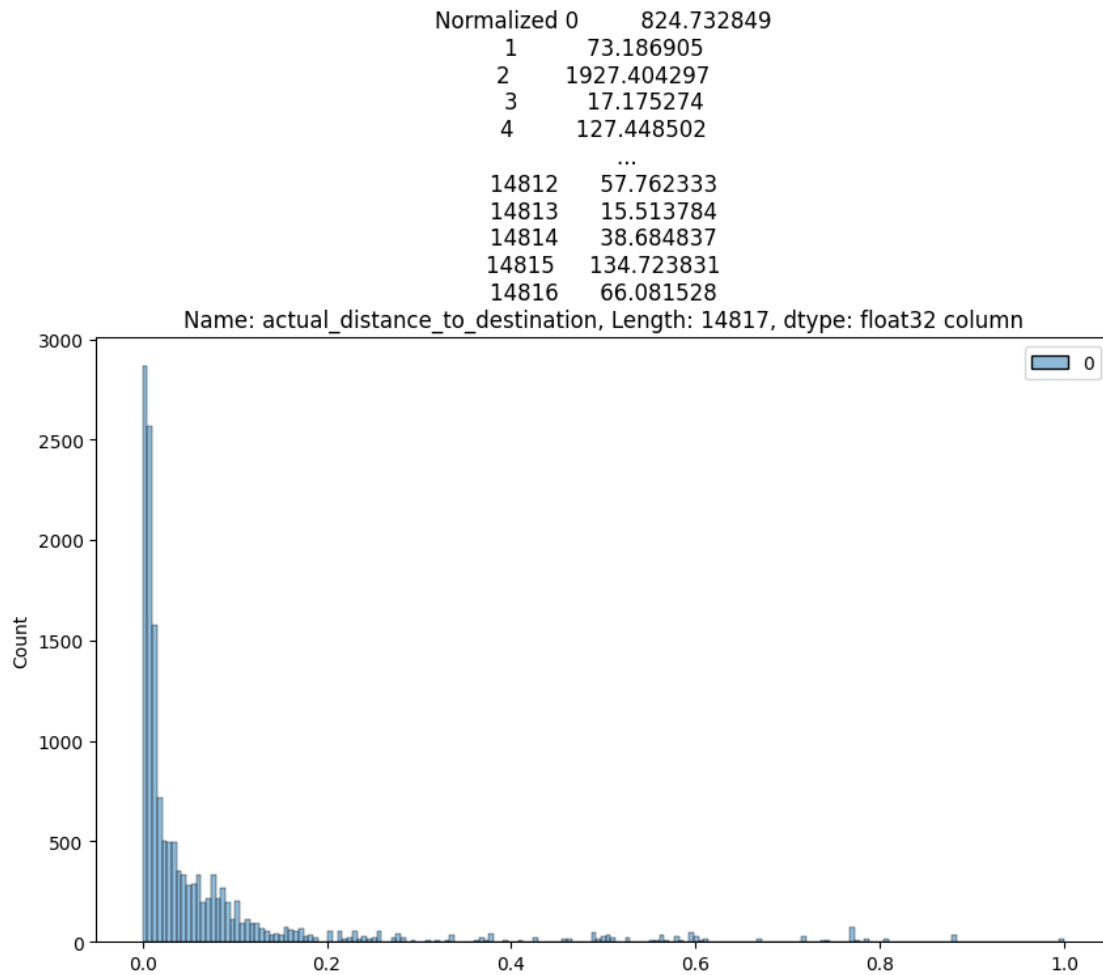
```
scaled = scaler.fit_transform(df2['start_scan_to_end_scan'].to_numpy().
    ↪reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Normalized {df2['start_scan_to_end_scan']} column")
plt.plot()
```

[162]: []



```
[163]: plt.figure(figsize = (10, 6))
scaler = MinMaxScaler()
scaled = scaler.fit_transform(df2['actual_distance_to_destination'].to_numpy().
    ↪reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Normalized {df2['actual_distance_to_destination']} column")
plt.plot()
```

[163]: []

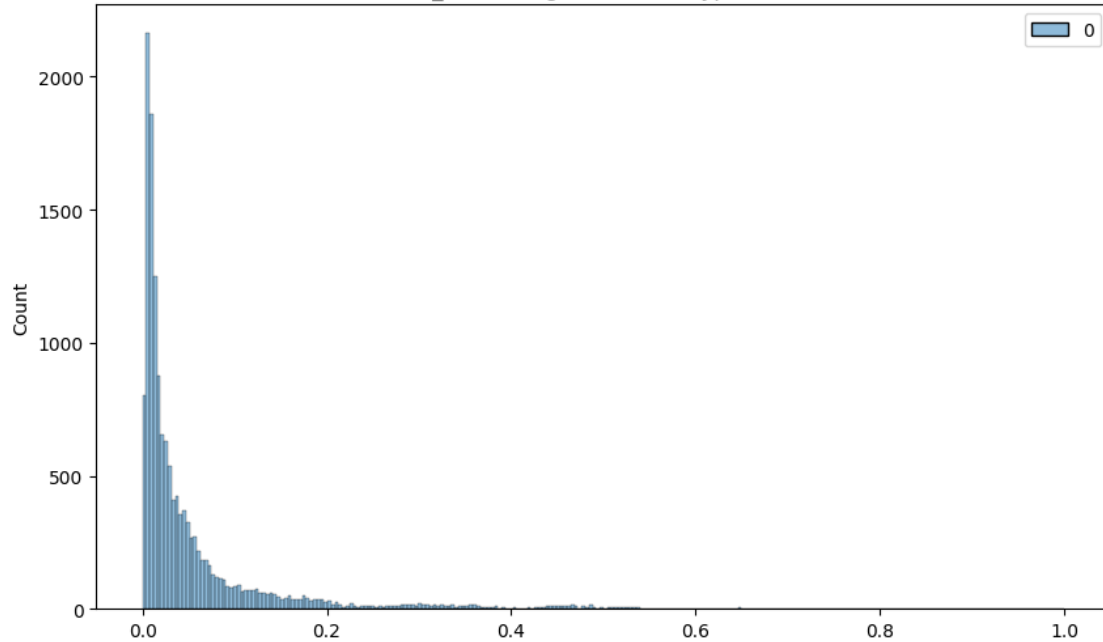


```
[164]: plt.figure(figsize = (10, 6))
scaler = MinMaxScaler()
scaled = scaler.fit_transform(df2['actual_time'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Normalized {df2['actual_time']} column")
plt.plot()
```

[164]: []

	Normalized 0	1562.0
1	143.0	
2	3347.0	
3	59.0	
4	341.0	
	...	
14812	83.0	
14813	21.0	
14814	282.0	
14815	264.0	
14816	275.0	

Name: actual_time, Length: 14817, dtype: float32 column

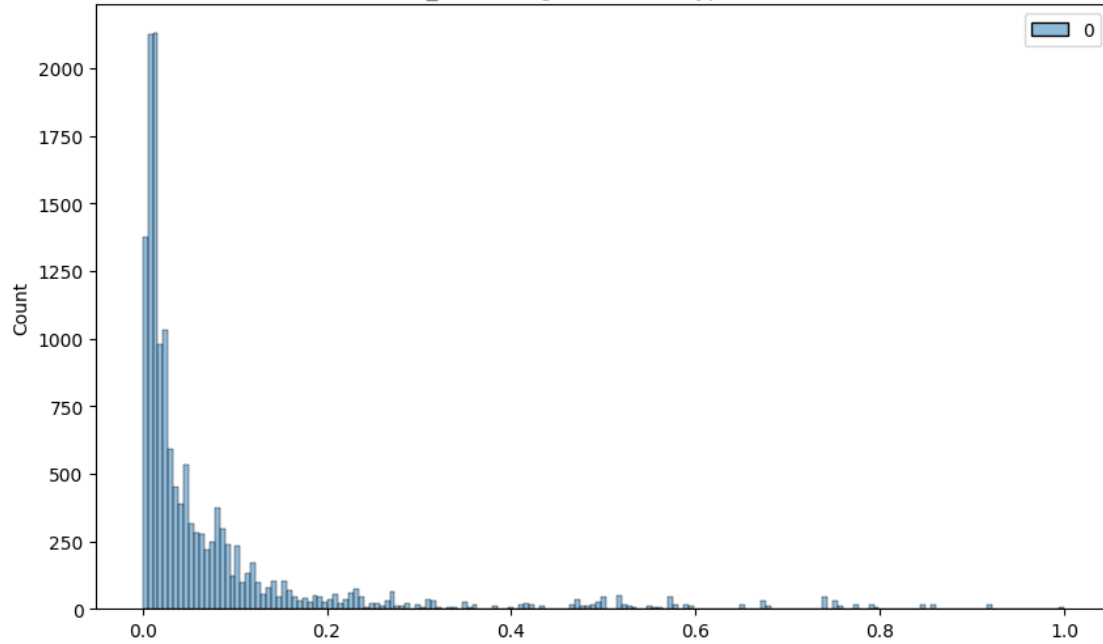


```
[165]: plt.figure(figsize = (10, 6))
scaler = MinMaxScaler()
scaled = scaler.fit_transform(df2['osrm_time'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Normalized {df2['osrm_time']} column")
plt.plot()
```

[165]: []

	Normalized 0	717.0
1	68.0	
2	1740.0	
3	15.0	
4	117.0	
	...	
14812	62.0	
14813	12.0	
14814	48.0	
14815	179.0	
14816	68.0	

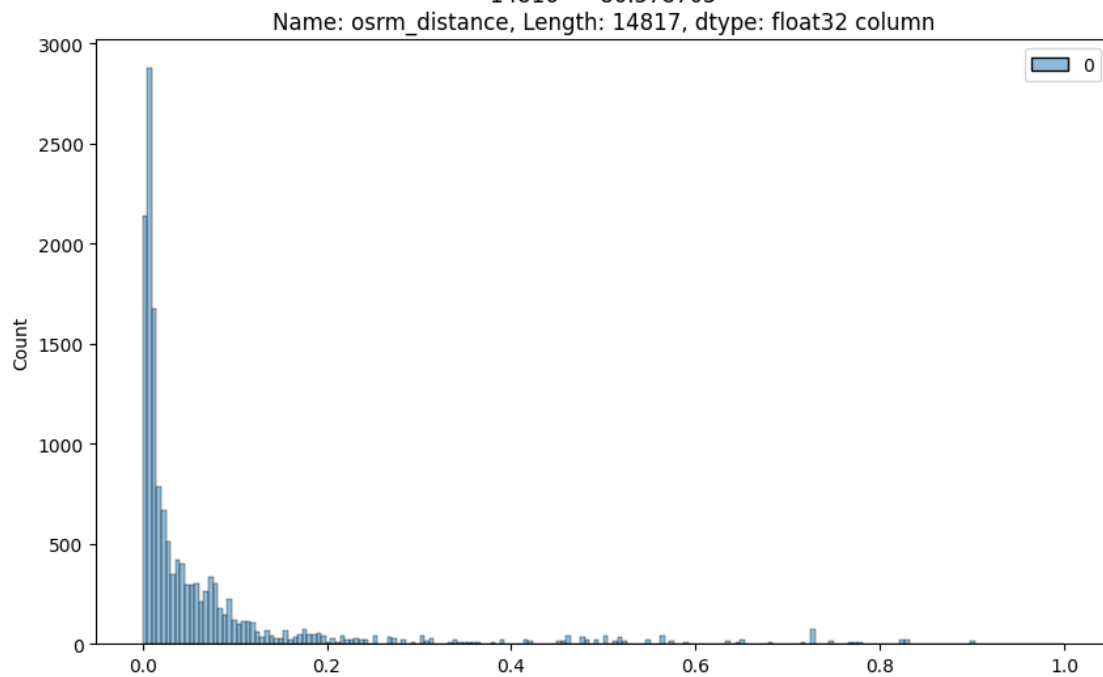
Name: osrm_time, Length: 14817, dtype: float32 column



```
[166]: plt.figure(figsize = (10, 6))
scaler = MinMaxScaler()
scaled = scaler.fit_transform(df2['osrm_distance'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Normalized {df2['osrm_distance']} column")
plt.plot()
```

[166]: []

Normalized	0	991.352295
1	85.111000	
2	2354.066650	
3	19.680000	
4	146.791794	
	...	
14812	73.462997	
14813	16.088200	
14814	58.903702	
14815	171.110306	
14816	80.578705	

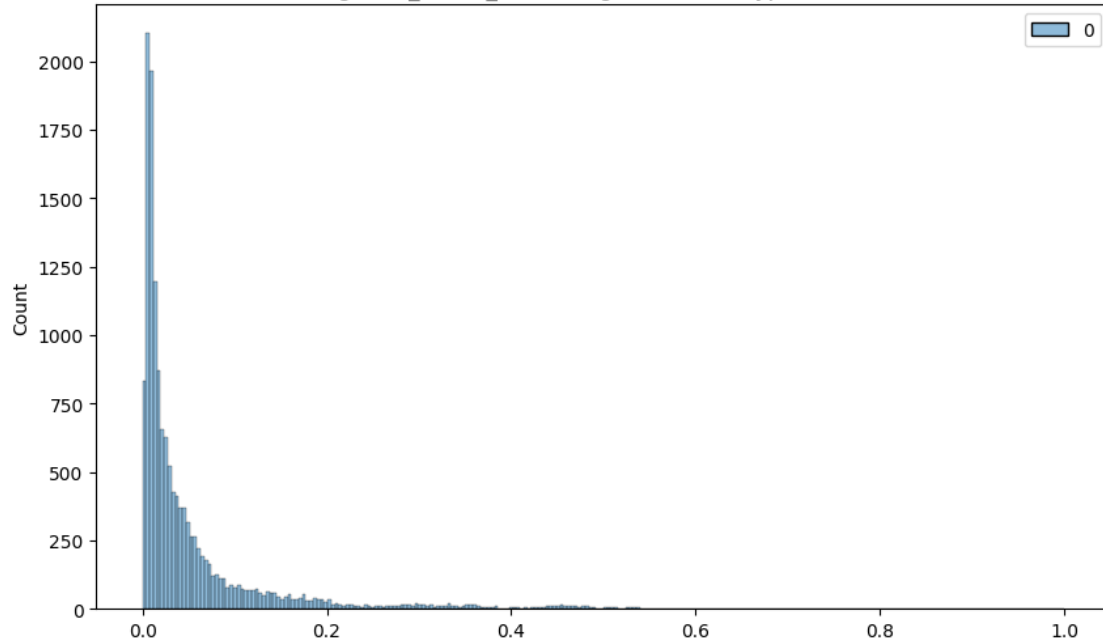


```
[167]: plt.figure(figsize = (10, 6))
scaler = MinMaxScaler()
scaled = scaler.fit_transform(df2['segment_actual_time'].to_numpy().reshape(-1,1))
sns.histplot(scaled)
plt.title(f"Normalized {df2['segment_actual_time']} column")
plt.plot()
```

[167]: []

	Normalized 0	1548.0
1	141.0	
2	3308.0	
3	59.0	
4	340.0	
	...	
14812	82.0	
14813	21.0	
14814	281.0	
14815	258.0	
14816	274.0	

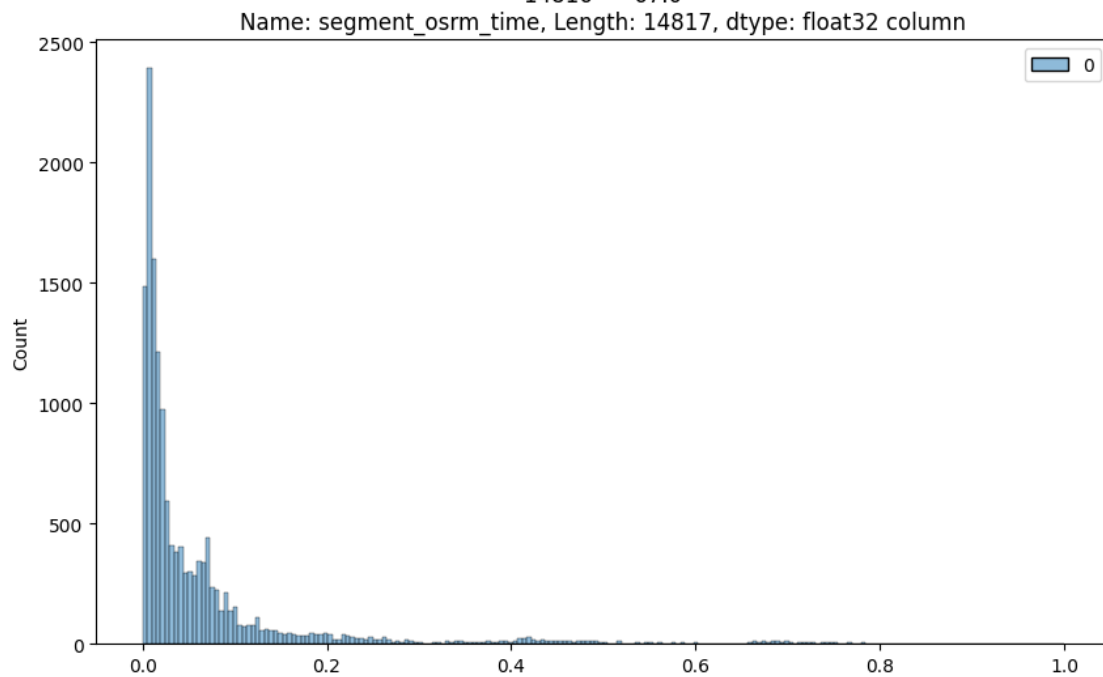
Name: segment_actual_time, Length: 14817, dtype: float32 column



```
[168]: plt.figure(figsize = (10, 6))
scaler = MinMaxScaler()
scaled = scaler.fit_transform(df2['segment_osrm_time'].to_numpy().reshape(-1,1))
sns.histplot(scaled)
plt.title(f"Normalized {df2['segment_osrm_time']} column")
plt.plot()
```

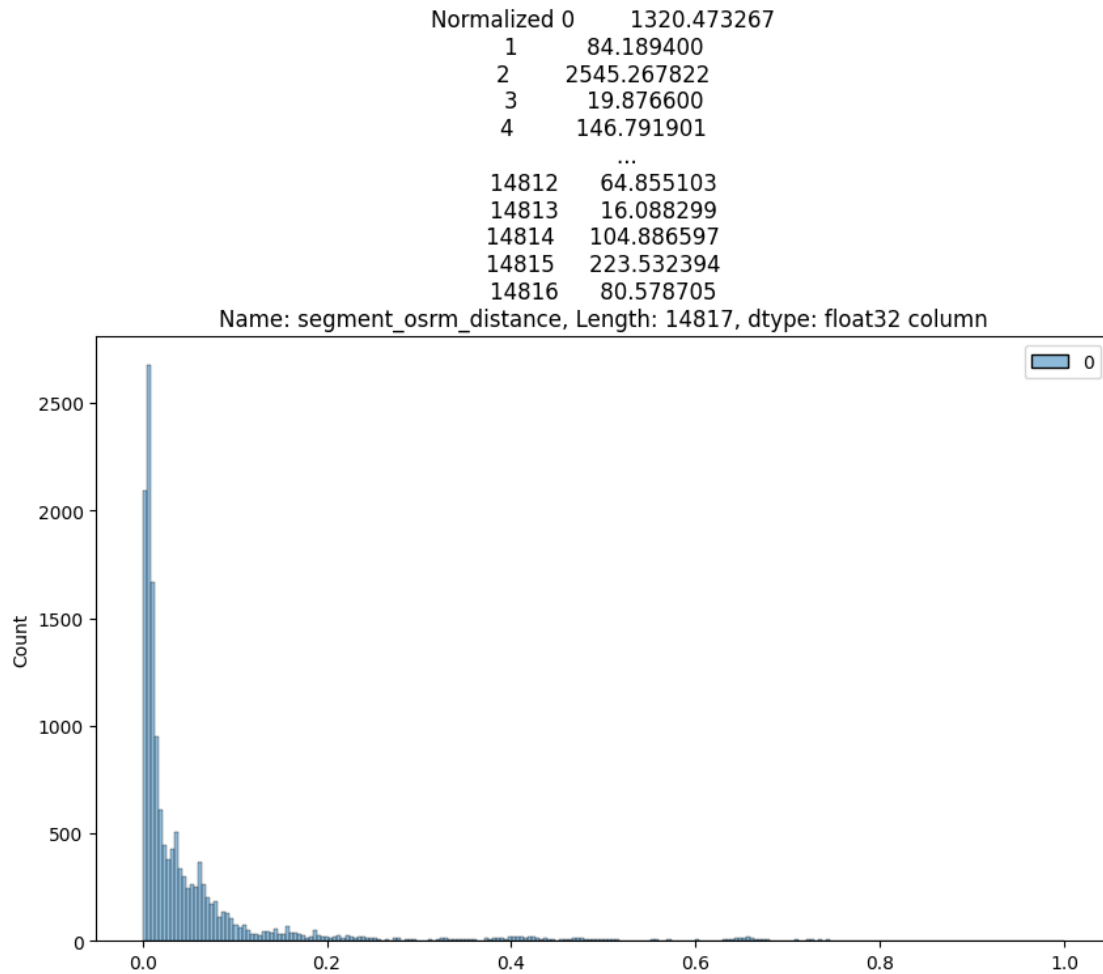
[168]: []

	Normalized 0	1008.0
1	65.0	
2	1941.0	
3	16.0	
4	115.0	
	...	
14812	62.0	
14813	11.0	
14814	88.0	
14815	221.0	
14816	67.0	



```
[169]: plt.figure(figsize = (10, 6))
scaler = MinMaxScaler()
scaled = scaler.fit_transform(df2['segment_osrm_distance'].to_numpy().
    ↪ reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Normalized {df2['segment_osrm_distance']} column")
plt.plot()
```

[169]: []



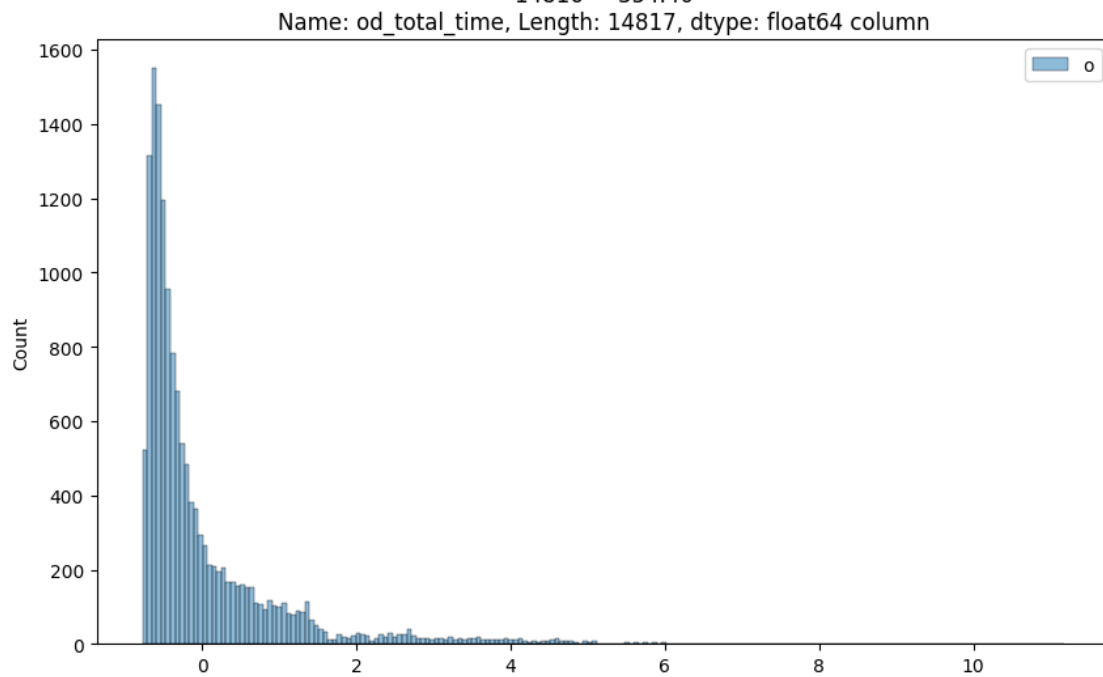
0.1.15 Column Standardization

```
[170]: from sklearn.preprocessing import StandardScaler
```

```
[171]: plt.figure(figsize = (10, 6))
# define standard scaler
scaler = StandardScaler()
# transform data
scaled = scaler.fit_transform(df2['od_total_time'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Standardized {df2['od_total_time']} column")
plt.legend('od_total_time')
plt.plot()
```

```
[171]: []
```

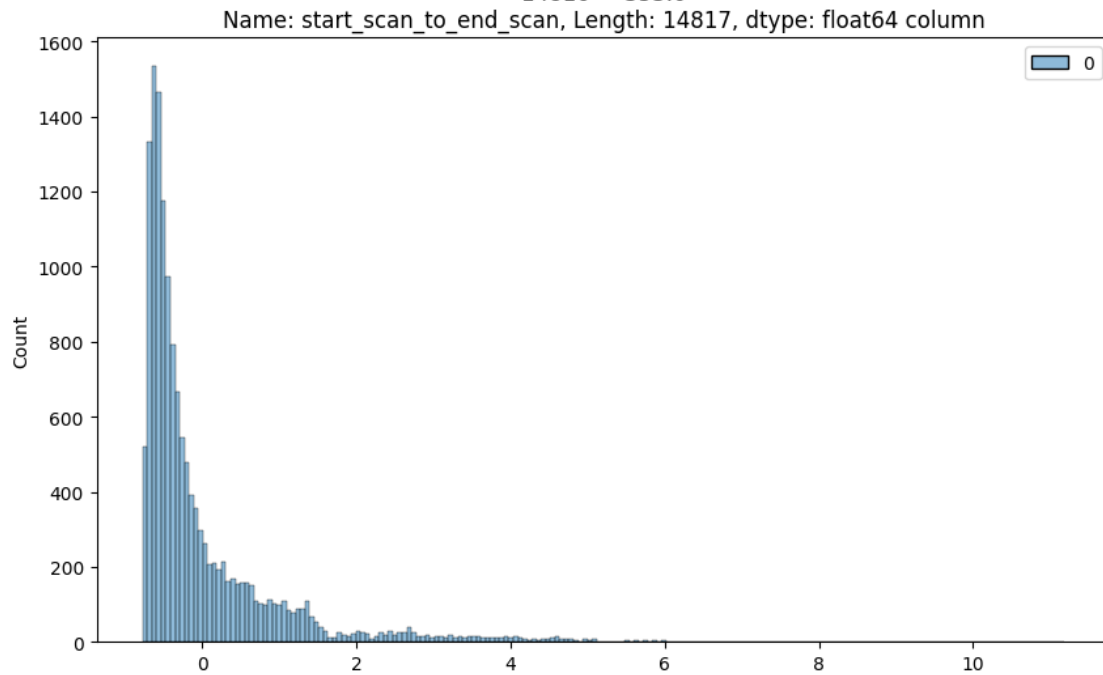
Standardized 0	2260.11
1	181.61
2	3934.36
3	100.49
4	718.34
	...
14812	258.03
14813	60.59
14814	422.12
14815	348.52
14816	354.40



```
[172]: plt.figure(figsize = (10, 6))
scaler = StandardScaler()
scaled = scaler.fit_transform(df2['start_scan_to_end_scan'].to_numpy().
    ↪ reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Standardized {df2['start_scan_to_end_scan']} column")
plt.plot()
```

[172]: []

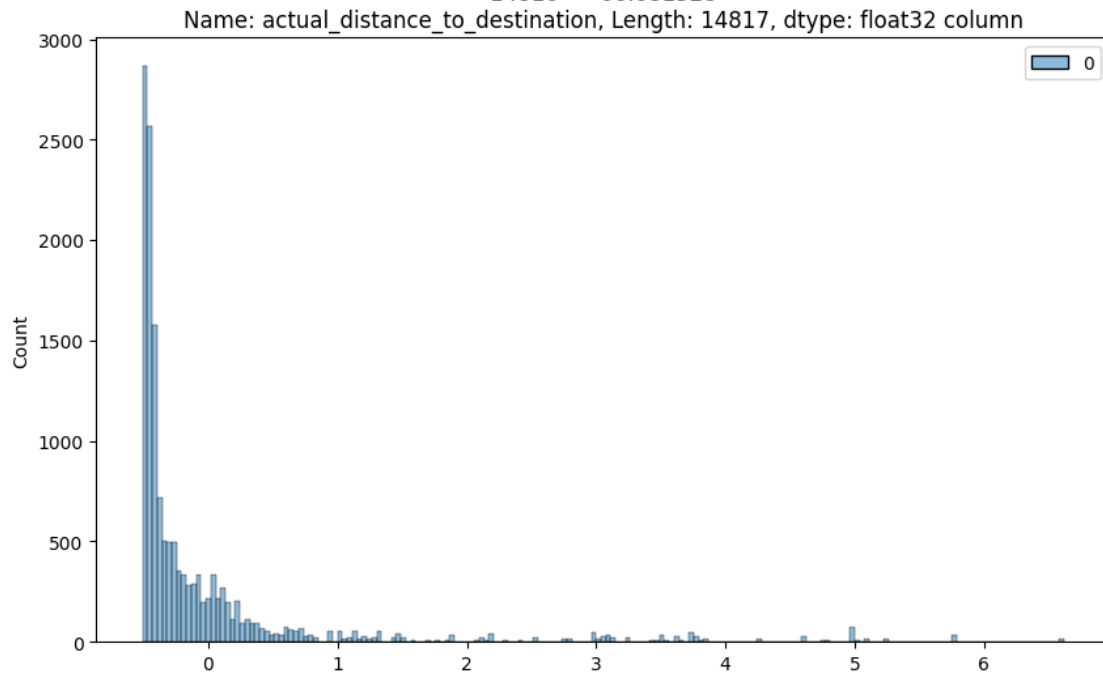
	Standardized 0	2259.0
1	180.0	
2	3933.0	
3	100.0	
4	717.0	
	...	
14812	257.0	
14813	60.0	
14814	421.0	
14815	347.0	
14816	353.0	



```
[173]: plt.figure(figsize = (10, 6))
scaler = StandardScaler()
scaled = scaler.fit_transform(df2['actual_distance_to_destination'].to_numpy().
    ↪ reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Standardized {df2['actual_distance_to_destination']} column")
plt.plot()
```

[173]: []

Standardized 0	824.732849
1	73.186905
2	1927.404297
3	17.175274
4	127.448502
	...
14812	57.762333
14813	15.513784
14814	38.684837
14815	134.723831
14816	66.081528

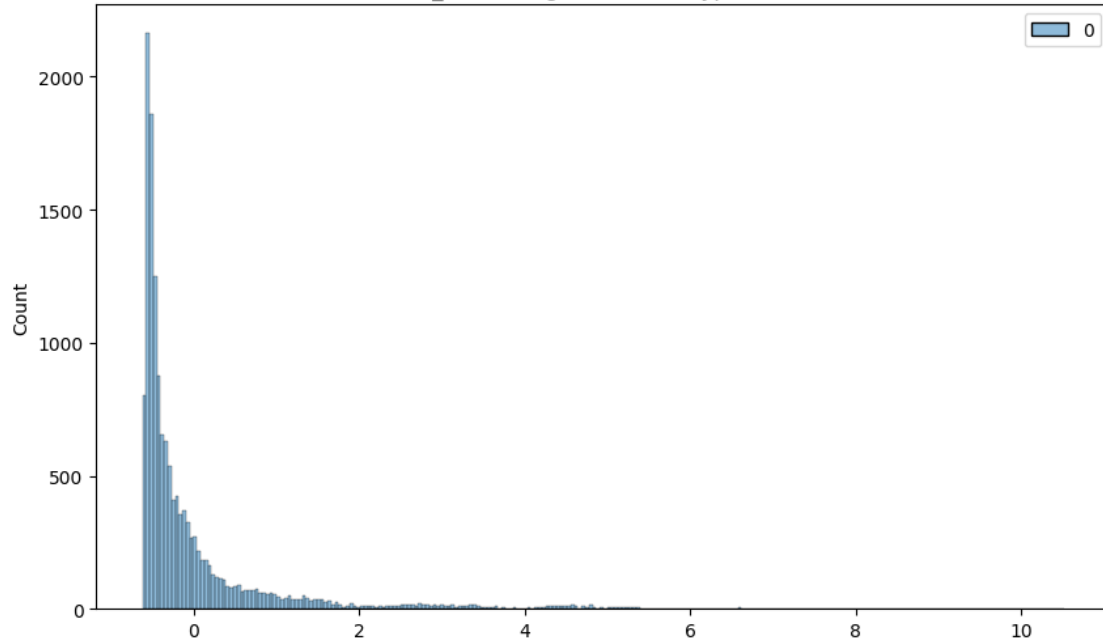


```
[174]: plt.figure(figsize = (10, 6))
scaler = StandardScaler()
scaled = scaler.fit_transform(df2['actual_time'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Standardized {df2['actual_time']} column")
plt.plot()
```

[174]: []

	Standardized 0	1562.0
1	143.0	
2	3347.0	
3	59.0	
4	341.0	
	...	
14812	83.0	
14813	21.0	
14814	282.0	
14815	264.0	
14816	275.0	

Name: actual_time, Length: 14817, dtype: float32 column

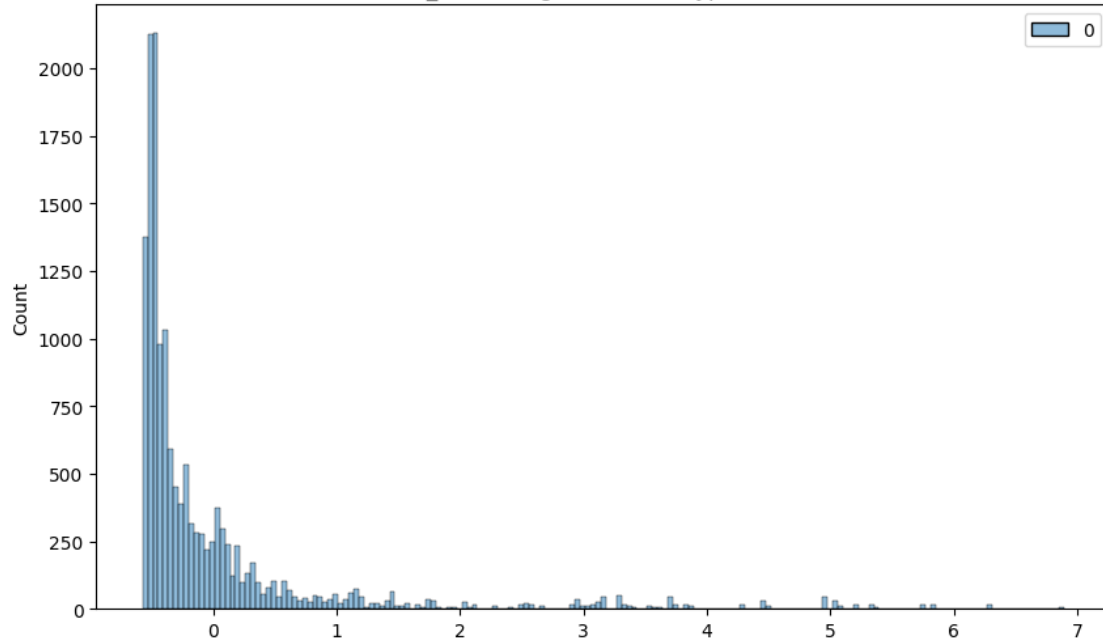


```
[175]: plt.figure(figsize = (10, 6))
scaler = StandardScaler()
scaled = scaler.fit_transform(df2['osrm_time'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Standardized {df2['osrm_time']} column")
plt.plot()
```

[175]: []

	Standardized 0	717.0
1	68.0	
2	1740.0	
3	15.0	
4	117.0	
	...	
14812	62.0	
14813	12.0	
14814	48.0	
14815	179.0	
14816	68.0	

Name: osrm_time, Length: 14817, dtype: float32 column



```
[176]: plt.figure(figsize = (10, 6))
scaler = StandardScaler()
scaled = scaler.fit_transform(df2['osrm_distance'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Standardized {df2['osrm_distance']} column")
plt.plot()
```

[176]: []

Standardized 0 991.352295

1 85.111000

2 2354.066650

3 19.680000

4 146.791794

...

14812 73.462997

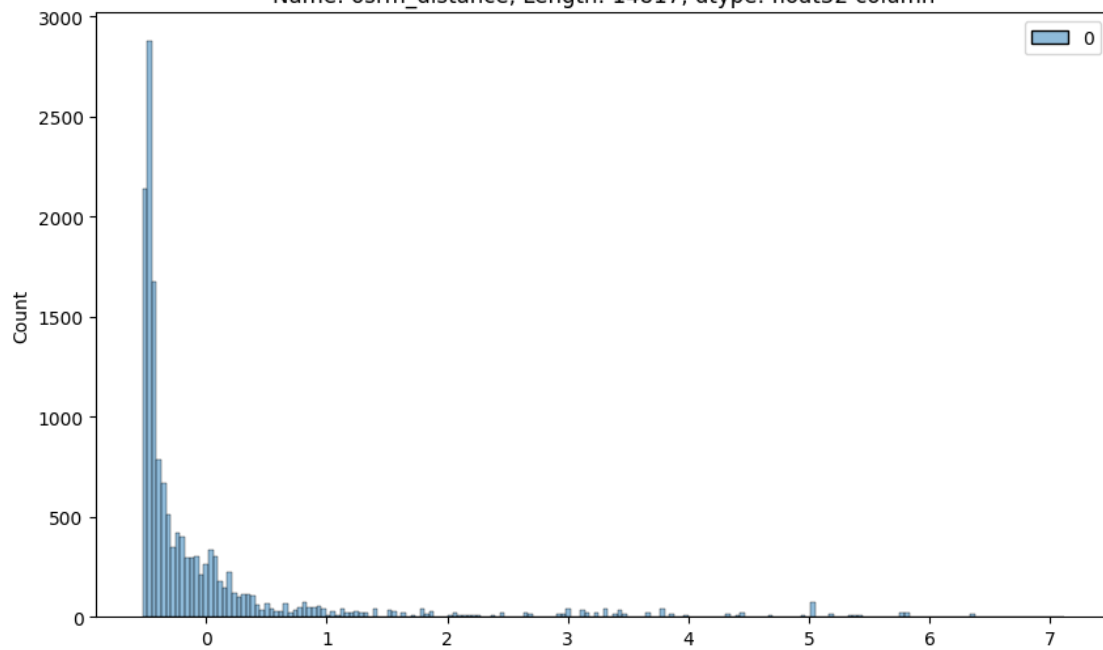
14813 16.088200

14814 58.903702

14815 171.110306

14816 80.578705

Name: osrm_distance, Length: 14817, dtype: float32 column

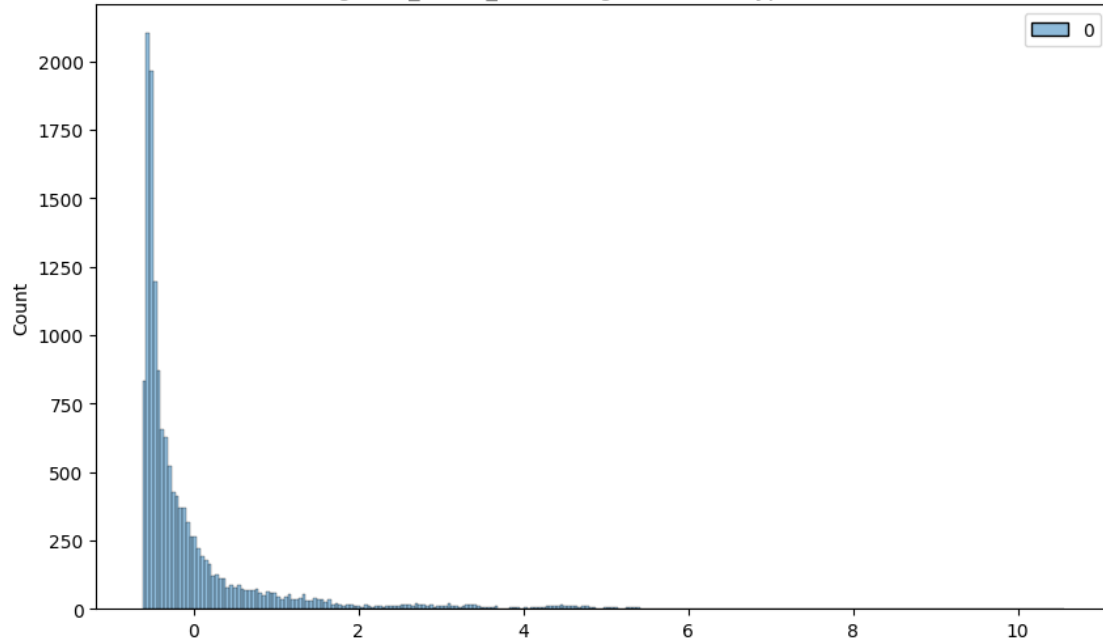


```
[177]: plt.figure(figsize = (10, 6))
scaler = StandardScaler()
scaled = scaler.fit_transform(df2['segment_actual_time'].to_numpy().reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Standardized {df2['segment_actual_time']} column")
plt.plot()
```

[177]: []

	Standardized 0	1548.0
1	141.0	
2	3308.0	
3	59.0	
4	340.0	
	...	
14812	82.0	
14813	21.0	
14814	281.0	
14815	258.0	
14816	274.0	

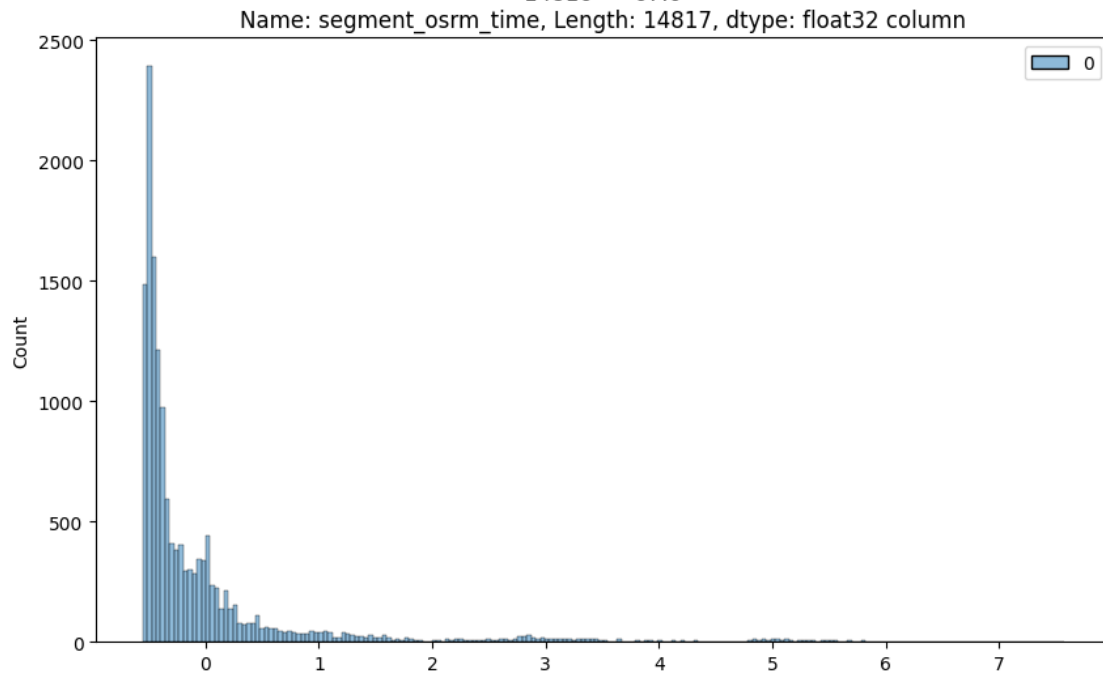
Name: segment_actual_time, Length: 14817, dtype: float32 column



```
[178]: plt.figure(figsize = (10, 6))
scaler = StandardScaler()
scaled = scaler.fit_transform(df2['segment_osrm_time'].to_numpy().reshape(-1,1))
sns.histplot(scaled)
plt.title(f"Standardized {df2['segment_osrm_time']} column")
plt.plot()
```

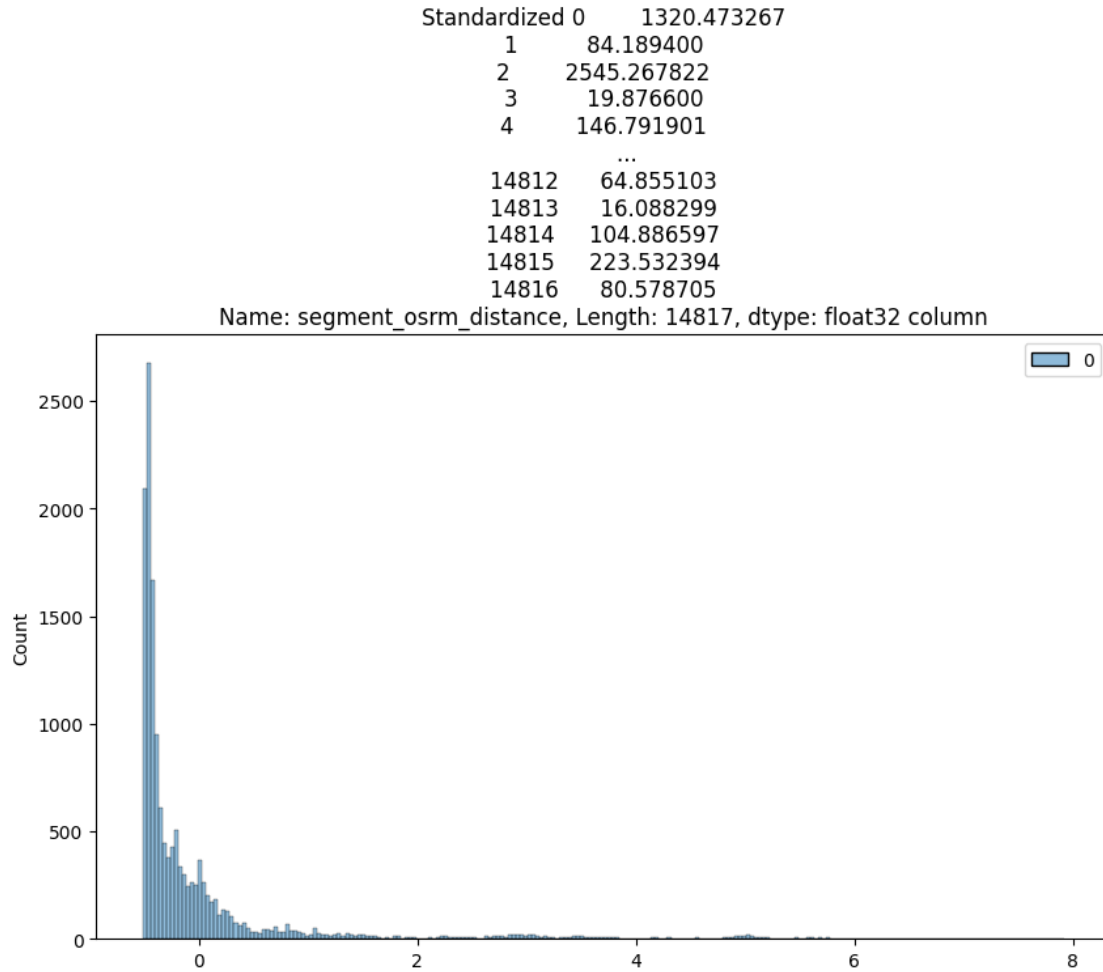
[178]: []

	Standardized 0	1008.0
1	65.0	
2	1941.0	
3	16.0	
4	115.0	
	...	
14812	62.0	
14813	11.0	
14814	88.0	
14815	221.0	
14816	67.0	



```
[179]: plt.figure(figsize = (10, 6))
scaler = StandardScaler()
scaled = scaler.fit_transform(df2['segment_osrm_distance'].to_numpy()
    ↪ reshape(-1, 1))
sns.histplot(scaled)
plt.title(f"Standardized {df2['segment_osrm_distance']} column")
plt.plot()
```

[179]: []



0.2 Business Insights|

- The data is given from the period ‘2018-09-12 00:00:16’ to ‘2018-10-08 03:00:24’.
- There are about 14817 unique trip IDs, 1508 unique source centers, 1481 unique destination_centers, 690 unique source cities, 806 unique destination cities.
- Most of the data is for testing than for training.
- ost common route type is Carting.
- The names of 14 unique location ids are missing in the data.
- The number of trips start increasing after the noon, becomes maximum at 10 P.M and then start decreasing.
- Maximum trips are created in the 38th week.
- Most orders come mid-month. That means customers usually make more orders in the mid of the month.

- Most orders are sourced from the states like Maharashtra, Karnataka, Haryana, Tamil Nadu, Telangana
- Maximum number of trips originated from Mumbai city followed by Gurgaon Delhi, Bengaluru and Bhiwandi. That means that the seller base is strong in these cities.
- Maximum number of trips ended in Maharashtra state followed by Karnataka, Haryana, Tamil Nadu and Uttar Pradesh. That means that the number of orders placed in these states is significantly high.
- Maximum number of trips ended in Mumbai city followed by Bengaluru, Gurgaon, Delhi and Chennai. That means that the number of orders placed in these cities is significantly high.
- Most orders in terms of destination are coming from cities like bengaluru, mumbai, gurgaon, bangalore, Delhi.
- Features start_scan_to_end_scan and od_total_time(created feature) are statistically similar.
- Features actual_time & osrm_time are statitically different.
- Features start_scan_to_end_scan and segment_actual_time are statistically similar.
- Features osrm_distance and segment_osrm_distance are statistically different from each other.
- Both the osrm_time & segment_osrm_time are not statistically same.

0.3 Recommendations

- The OSRM trip planning system needs to be improved. Discrepancies need to be catered to for transporters, if the routing engine is configured for optimum results.
- osrm_time and actual_time are different. Team needs to make sure this difference is reduced, so that better delivery time prediction can be made and it becomes convenient for the customer to expect an accurate delivery time.
- The osrm distance and actual distance covered are also not same i.e. maybe the delivery person is not following the predefined route which may lead to late deliveries or the osrm devices is not properly predicting the route based on distance, traffic and other factors. Team needs to look into it.
- Most of the orders are coming from/reaching to states like Maharashtra, Karnataka, Haryana and Tamil Nadu. The existing corridors can be further enhanced to improve the penetration in these areas.
- Customer profiling of the customers belonging to the states Maharashtra, Karnataka, Haryana, Tamil Nadu and Uttar Pradesh has to be done to get to know why major orders are coming from these atates and to improve customers' buying and delivery experience.
- From state point of view, we might have very heavy traffic in certain states and bad terrain conditions in certain states. This will be a good indicator to plan and cater to demand during peak festival seasons.

[]: